Dotter: Mathematical Analysis and Implementation Guide

Oscar Laird

March 2026

Contents

Introduction

Dotter is a text-entry technology that allows users to type over a 1-bit channel. It allows a simple input mechanism such as blinking or pressing a single button to be used to type. Dotter milks as much information out of that 1-bit channel as possible, allowing an experienced user to type standard english text at speeds of 20 words per minute only by blinking.

> **Dotter only has one rule:** Identify the longest visible prefix of your target text and synchronize your signal to its timer.

Dotter was inspired by the project "Dasher" [2], a text-entry technology developed by David MacKay's group at the University of Cambridge that took inspiration from arithmetic coding to map the space of all possible text sequences to the unit interval. Assuming a version of Fitt's law, Dasher is an optimal text-entry technology for 1-dimensional continuous (pointing) input.

This document is a mathematical analysis of Dotter as well as an implementation guide, describing the architecture and key algorithms of Dotter. Chapter 1 describes the architecture of Dotter, outlining its key data structures and algorithms. The succeeding chapters describe individual components of Dotter. The final chapter gives theory and advice on how to learn to type with Dotter. (If the person reading this only wants to learn how to use Dotter, they should skip to the final chapter.)

The name "Dotter" is a nod the most famous 1-bit text-entry system, telegraphy, which consisted of "dots" and "dashes." The name "Dasher" was already taken, so I went with this.

Chapter 1

Architecture

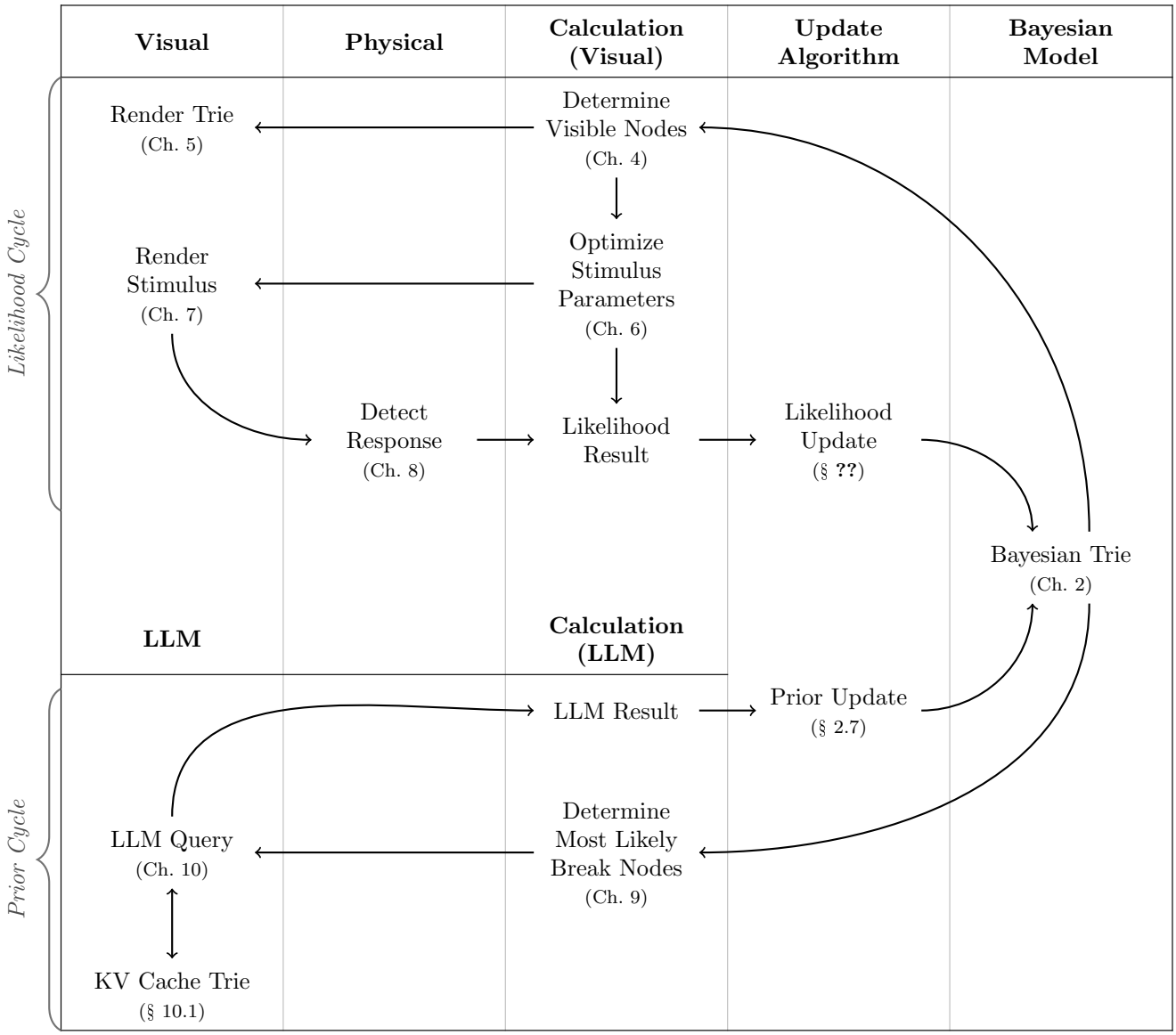


Table 1.1: Architecture of Dotter

Table 1.1 shows the architecture of Dotter. At its heart is the "bayesian trie" (chapter 2) in the rightmost column, which is a character-based Bayesian prefix trie which maintains a calculation of the posterior probability of every prefix. The other elements of the system may be separated into the Likelihood Cycle and the Prior Cycle.

Each element in Table 1.1 is discussed in detail in its own chapter or section.

1.1 The Likelihood Cycle

The Likelihood Cycle refers to the series of steps by which we elicit a signal from the user and then interpret that signal to update our posterior probabilities. It has seven steps,

1. **Determine Visible Nodes** (chapter 4): we determine which prefixes should be visually displayed by checking which prefixes have a posterior probability exceeding the visibility threshold (e.g. 2%).
2. **Render Trie** (chapter 5): here we draw the visible prefixes to the screen, connecting parent nodes to their children with smooth curves.
3. **Optimize Stimulus Parameters** (chapter 6): in this step we assign timers to every prefix. We want to maximize the information gain from the user's signal which is accomplished by making the timers as far apart from each other as possible (in a formal sense). This helps to make the user's selection unambiguous.
4. **Render Stimulus** (chapter 7): we draw the timers to the screen.
5. **Detect Response** (chapter 8): we detect the user's signal. It is trivial to detect a keypress; detecting the exact timing of a blink is a more sophisticated problem.
6. **Likelihood Result**: we compare the detected user signal to the timers that were assigned in step 3. This gives us a likelihood score for each prefix.
7. **Likelihood Update** (§ **??**): we update the trie to reflect the received likelihoods. Performing this update in a lazy and efficient manner is a central technical challenge of Dotter. (This is an "update" in the traditional bayesian sense.)

1.2 The Prior Cycle

The Prior Cycle refers to the series of steps by which we query a large language model to refine our prior probabilities and incorporate those results into our posterior beliefs. It has four steps,

1. **Determine Most Likely Break Nodes** (chapter 9): we determine which nodes are most likely to a break between tokens in the tokenization of the user's target text.
2. **LLM Query** (chapter 10): we query the large language model to predict the distribution over the next token for the prefixes identified in the previous step, making use of the Key-Value Cache Trie (§ 10.1).
3. **LLM Result**: we receive the result from the language model.
4. **Prior Update** (§ 2.7): we update the trie to reflect the received prior probabilities. Performing this update in a lazy and efficient manner is a central technical challenge of Dotter. (This is not truly an "update" in the traditional bayesian sense, since normally our priors are fixed.)

1.3 Client-Server Communication

A web-based client is responsible for the likelihood cycle, while a server with a large language model handles the prior cycle. Since these two cycles must run independently, it is necessary for both the client and the server to maintain a copy of the trie.

The data designated "Likelihood Result" and "LLM Result" are exchanged between the client and the server, with each side updating its copy of the trie in response to the stream of these results.

The two sides will not necessarily receive this stream of results in the same order, but an important correctness property of our update algorithms is that the state of the trie does not depend on the order in

which the updates are performed.

Chapter 2

The Bayesian Trie

Dotter must determine the most likely string that the user is typing, based on observation of the user's signals. To accomplish this, Dotter maintains a trie indicating the posterior probability that any given prefix is a prefix of the user's target string. To compare strings, we use the inequality notation $x \leq y$ to mean that the string x is a prefix of the string y. Letting X (an unknown random variable) be the user's target string and D be the user's signals, then the posterior probability that a given string, a, is a prefix of X is given by:

$$P(a \leq X|D) = \frac{P(D|a \leq X)P(a \leq X)}{P(D)}.$$

In practice, we only keep track of the unnormalized posterior probabilities, $P(D|a \leq X)P(a \leq X)$. However, since the root node, ϵ, is the empty string, we have $P(\epsilon \leq X) = 1$ and thus the unnormalized posterior probability of the root node is $P(D)$, i.e., the normalizing constant. Thus if we know the unnormalized posterior probabilities in our trie, we can convert these to normalized posterior probabilities by dividing by the root node's unnormalized posterior probability.

The posterior probabilities are derived from the prior probabilities and the likelihoods. In Dotter, both are subject to updates. The likelihood is updated as we receive signals from the user and the prior is updated as our language model refines its predictions.

Our analysis has six parts:

1. **Core Algorithms** (section 2.1): we present the core algorithms for lazy calculation of the unnormalized posteriors.
2. **Likelihood Tracking** (section 2.2): we present the algorithm for lazy likelihood tracking.
3. **Character Priors from Token Models** (section 2.3): we present the algorithm for converting a token-based prior model to a character-based prior model.
4. **Prior Tracking** (section 2.4): we present the algorithm for calculating priors.
5. **Maximum Truncation Compatible Descendant Likelihood** (section 2.5): we present the algorithm for tracking "maximum truncation compatible descendant likelihoods".
6. **The Complete Algorithms** (section **??**): we present the complete algorithms.

2.1 Lazy Posterior Calculation: The Core Algorithms

Definition 2.1 (Alphabet)**.** *An alphabet is a finite set.*

Definition 2.2 (Character)**.** *A character is an element of an alphabet.*

Definition 2.3 (String)**.** *A string is a sequence of characters.*

Throughout this chapter, we will let Λ denote our alphabet which must include at least the two special characters: space, σ, and stop, ω. The set of all strings over this alphabet will be denoted S.

Definition 2.4 (Substring)**.** *We use the notation $s[i:j]$ to represent the subsequence of characters of s from index i to index $j-1$. We also allow the notation $s[:-k]$ to represent the subsequence of characters of s from index 0 to index $|s|-k-1$.*

Definition 2.5 (Empty String)**.** *The empty string is the string of length 0, denoted ϵ.*

Definition 2.6 (String Inequality)**.** *We say that a string s is a prefix of a string t, and write this as $s \leq t$, if and only if there exists an integer k such that $s = t[0:k]$.*

Definition 2.7 (String Subtraction)**.** *String subtraction refers to prefix removal. $x - y$ is defined as $x[|y|:]$. It is only defined if $y \leq x$.*

Definition 2.8 (Properly Terminated String)**.** *A string h is properly terminated if and only if $h[i] = \omega \iff i = |h| - 1$. An unterminated string is one that does not contain the stop character. An "at most properly terminated string" is one that is either properly terminated or unterminated.*

In this chapter, the set of all properly terminated strings will be denoted H.

Definition 2.9 (Prefix Code)**.** *A prefix code is a set of strings such that no string is a prefix of another string in the set, i.e., A set C is a prefix code if and only if,*

$$\nexists c_1, c_2 \in C, c_1 < c_2$$

Definition 2.10 (Complete Prefix Code)**.** *A complete prefix code, C, is a prefix code that covers the entire tree, i.e.,*

$$\forall x \in S, \exists c \in C, x \leq c \vee c \leq x$$

Definition 2.11 (Refinement)**.** *A prefix code, A, is a refinement of a prefix code, B, if every element of A is an extension of some element of B, i.e.,*

$$\forall a \in A, \exists b \in B, a \geq b$$

If this is the case, we write $A \geq B$.

Refinement is a partial order over complete prefix codes. These prefix codes form a lattice, so we may define the meet and join of two prefix codes.

Definition 2.12 (Meet)**.** *The meet of two prefix codes, A and B, is the greatest lower bound of A and B, i.e., i.e., it is the set of minimal elements of their intersection. We write this as $A \wedge B$.*

Definition 2.13 (Join)**.** *The join of two prefix codes, A and B, is the least upper bound of A and B, i.e., it is the set of maximal elements of their union. We write this as $A \vee B$.*

Definition 2.14 (Element Refinement)**.** *We overload the comparison operators when one element is a string and the other is a complete prefix code, to mean that there is an element of the complete prefix code that satisfies the comparison operator. For example, $x \leq C$ means that x is a prefix of some element of C, whereas $x > C$ means that x is a proper extension of some element of C.*

Proposition 2.1. *The set of all properly terminated strings, H, is a complete prefix code.*

Proof. We first show that H is a prefix code. We proceed by contradiction. Suppose there exist $h_1, h_2 \in H$ such that $h_1 < h_2$. Because h_1 is a strict prefix of h_2, it must be that $|h_1| < |h_2|$. Furthermore, the characters of h_2 must perfectly match h_1 up to index $|h_1| - 1$. Thus,

$$h_2[|h_1| - 1] = h_1[|h_1| - 1].$$

Because $h_1 \in H$, it is properly terminated, meaning its final character is the stop character: $h_1[|h_1|-1] = \omega$. Therefore, $h_2[|h_1| - 1] = \omega$. However, because $h_2 \in H$, the stop character can only appear at its final index $|h_2| - 1$. This implies:

$$|h_1| - 1 = |h_2| - 1 \implies |h_1| = |h_2|,$$

which contradicts the strict prefix requirement $|h_1| < |h_2|$. Thus, no such pair h_1, h_2 can exist, and H is a prefix code.

We now show that H is a complete prefix code. Let $x \in S$ be any string. We show this by cases constructively:

(a) If ω is in x, then let i be the index of its first occurrence. Then $x[:i+1] \in H$ and $x[:i+1] \leq x$.
(b) If ω is not in x, then $x + \omega \in H$ and $x < x + \omega$.

□

Definition 2.15 (Likelihood Function)**.** *A likelihood function is a mapping from properly terminated strings to non-negative real numbers, $L_n : H \to \mathbb{R}^+$.*

Definition 2.16 (Prior Function)**.** *A prior function is a mapping from properly terminated strings to non-negative real numbers, $P_m : H \to \mathbb{R}^+$, satisfying*

$$\sum_{h \in H} P_m(h) = 1$$

Definition 2.17 (Branch Prior Function)**.** *We define the branch prior function, $P_m^{br}(x)$, of an at most properly terminated string x, as,*

$$P_m^{br}(x) = \sum_{\substack{h \in H \\ h \geq x}} P_m(h)$$

Terminology Note: The branch prior function corresponds to our usual notion of a prior over string prefixes, but we choose to name it explicitly as "branch prior" and to restrict our definition of the domain of a "prior function" to properly terminated strings, excluding their prefixes. We make this choice to keep a close correspondence between the definitions of prior and likelihood functions.

Definition 2.18 (Likelihood Sequence)**.** *A likelihood sequence is a sequence of likelihood functions, $\{L_n\}$, with L_0 initialized to 1, i.e.,*

$$L_0(h) = 1 \quad \forall h \in H$$

Definition 2.19 (Prior Sequence)**.** *A prior sequence is a sequence of prior functions $\{P_n\}$.*

Definition 2.20 (Delta Digest)**.** *A delta digest, (C, Δ), is a prefix code, C, and a mapping from that prefix code to non-negative real numbers, Δ, i.e.,*

$$\Delta : C \to \mathbb{R}^+$$

Definition 2.21 (Prefix Code Truncation)**.** *The truncation of a string x under a prefix code C refers to the element $c \in C$ such that $c \leq x$. If no such element exists, then the truncation is undefined. We denote the truncation of x under C as $\lfloor x \rfloor_C$.*

Definition 2.22 (Likelihood Delta Digest Sequence)**.** *The likelihood delta digest sequence corresponding to a likelihood sequence $\{L_n\}$ is a sequence of delta digests $\{(C_n^L, \Delta_n^L)\}$, that satisfies:*

$$\forall h \in H, L_n(h) = L_{n-1}(h)\Delta_n^L(\lfloor h \rfloor_{C_n^L})$$

for the truncation operation to always be defined we require that $C_n^L \leq H$.

Definition 2.23 (Prior Delta Digest Sequence)**.** *This is defined identically, i.e., the prior delta digest sequence corresponding to a prior sequence $\{P_n\}$ is a sequence of delta digests $\{(C_n^P, \Delta_n^P)\}$, that satisfies:*

$$\forall h \in H, P_n(h) = P_{n-1}(h)\Delta_n^P(\lfloor h \rfloor_{C_n^P})$$

for the truncation operation to always be defined we require that $C_n^P \leq H$.

Lemma 2.1 (Likelihood Delta Product)**.**

$$L_N(h) = \prod_{1 \leq i \leq N} \Delta_i^L(\lfloor h \rfloor_{C_i^L})$$

Proof. Fix any $h \in H$. By definition of a likelihood delta digest sequence, for each $i \in \{1, \ldots, N\}$,

$$L_i(h) = L_{i-1}(h)\, \Delta_i^L\Big(\lfloor h \rfloor_{C_i^L}\Big).$$

Applying this recursively from $i = N$ down to $i = 1$ gives

$$L_N(h) = L_0(h) \prod_{1 \le i \le N} \Delta_i^L\Big(\lfloor h \rfloor_{C_i^L}\Big).$$

By definition of the likelihood sequence, $L_0(h) = 1$, so

$$L_N(h) = \prod_{1 \le i \le N} \Delta_i^L\Big(\lfloor h \rfloor_{C_i^L}\Big),$$

as claimed. □

Lemma 2.2 (Eventually Constant Likelihoods)**.** *Let $f \in \bigvee_{i=1}^N C_i^L$ be a node in the cumulative likelihood update frontier. Then every properly terminated descendant $h \in H$ of f has the same likelihood at each time $n \le N$. More precisely,*

$$\forall n \le N, \forall h \in H,\ h \ge f \implies L_n(h) = \prod_{1 \le i \le n} \Delta_i^L(\lfloor f \rfloor_{C_i^L})$$

Proof. From lemma 2.1,

$$L_n(h) = \prod_{1 \le i \le n} \Delta_i^L(\lfloor h \rfloor_{C_i^L}).$$

We must show that $\lfloor h \rfloor_{C_i^L} = \lfloor f \rfloor_{C_i^L}$ for all $1 \le i \le n$.

If $f \in \bigvee_{i=1}^N C_i^L$, then for every $i \in \{1, \ldots, n\}$, there exists some $c_i \in C_i^L$ such that $c_i \le f$. Because c_i is a prefix of f, it is exactly the truncation of f under C_i^L, so $c_i = \lfloor f \rfloor_{C_i^L}$.

By hypothesis, $h \ge f$. Since the prefix relation is transitive, $c_i \le f$ and $f \le h$ imply $c_i \le h$. Because C_i^L is a prefix code, c_i is the unique element in C_i^L that is a prefix of h. Therefore, $\lfloor h \rfloor_{C_i^L} = c_i = \lfloor f \rfloor_{C_i^L}$. Substituting this into the product yields the result. □

Definition 2.24 (Unnormalized Posterior Function)**.** *The unnormalized posterior function is a mapping from at most properly terminated strings to non-negative real numbers given by*

$$Z_{n,m}(x) = \sum_{\substack{h \in H \\ h \ge x}} L_n(h) P_m(h)$$

Proposition 2.2. *Before any likelihood updates, the unnormalized posterior of the root node is 1:*

$$\begin{aligned} Z_{0,m}(\epsilon) &= \sum_{\substack{h \in H \\ h \ge \epsilon}} L_0(h) P_m(h) \\ &= \sum_{h \in H} L_0(h) P_m(h) \\ &= \sum_{h \in H} 1 \cdot P_m(h) && \textit{(by definition 2.18)} \\ &= 1 && \textit{(by definition 2.16)} \end{aligned}$$

Lemma 2.3 (Posterior Equals Sum of Children)**.** *if x is an unterminated string, then*

$$Z_{n,m}(x) = \sum_{\lambda \in \Lambda} Z_{n,m}(x + \lambda)$$

Proof. By definition,

$$Z_{n,m}(x) = \sum_{\substack{h \in H \\ h \geq x}} L_n(h) P_m(h).$$

Since x is an unterminated string, it does not contain the stop character. Because all strings in H are properly terminated and therefore end with the stop character, $x \notin H$. Thus, any properly terminated string $h \in H$ that has x as a prefix must be strictly longer than x ($|h| > |x|$).

Therefore, the character in h immediately following the prefix x must be some character $\lambda \in \Lambda$. We can partition the set of strings $\{h \in H \mid h \geq x\}$ into mutually disjoint subsets based on this next character λ:

$$\{h \in H \mid h \geq x\} = \bigcup_{\lambda \in \Lambda} \{h \in H \mid h \geq x + \lambda\}.$$

Because these subsets are disjoint, we can split the summation over them:

$$\begin{aligned} Z_{n,m}(x) &= \sum_{\lambda \in \Lambda} \sum_{\substack{h \in H \\ h \geq x+\lambda}} L_n(h) P_m(h) \\ &= \sum_{\lambda \in \Lambda} Z_{n,m}(x + \lambda). \end{aligned}$$

□

We may now introduce the datastructure at the core of Dotter, the Bayesian Trie. The Bayesian Trie maintains a calculation of the unnormalized posterior probability while allowing for the lazy application of likelihood and prior updates.

Definition 2.25 (Bayesian Trie Node)**.** *A Bayesian trie node x associated with a string $s \in S$ is a data structure containing the tuple (n, m, Z), accessed via fields:*

- *$x.n_Z \in \mathbb{N}$, the "likelihood time" of the posterior calculation of the node.*
- *$x.m_Z \in \mathbb{N}$, the "prior time" of the posterior calculation of the node.*
- *$x.Z \in \mathbb{R}^+$, a computed quantity which should be equal to the unnormalized posterior of the node evaluated at those times, i.e., $x.Z = Z_{x.n_Z, x.m_Z}(s)$.*

Definition 2.26 (Valid Node)**.** *A node, x, is called valid at time (N_Z, M_Z) if it satisfies one of the following two conditions:*

1. ***Constant Likelihood**: x lies beyond the cumulative likelihood update frontier, i.e., $x \geq \bigvee_{i=1}^{N_Z} C_i^L$*
2. ***No Reweighting**: The likelihoods and the priors of descendants have changed by a constant factor since the node's time, i.e., $x \geq \bigvee_{i=x.n_Z+1}^{N_Z} C_i^L$ and $x \geq \bigvee_{i=x.m_Z+1}^{M_Z} C_i^P$*

Lemma 2.4 (Current Nodes are Valid)**.** *A node is always valid at its own time. I.e., $(x.n_Z, x.m_Z) = (N_Z, M_Z)$ implies that x is valid at time (N_Z, M_Z).*

Proof. Such a node satisfies the "No Reweighting" condition of validity. Because $(x.n_Z, x.m_Z) = (N_Z, M_Z)$, the two joins are joins over empty sets and thus equal to the root node. Hence, $x \geq \bigvee_{i=x.n_Z+1}^{N_Z} C_i^L$ and $x \geq \bigvee_{i=x.m_Z+1}^{M_Z} C_i^P$. □

Definition 2.27 (Bayesian Trie)**.** *A Bayesian Trie is a trie of Bayesian trie nodes along with a global tuple describing the current time, (N_Z, M_Z).*

Definition 2.28 (Valid Bayesian Trie)**.** *A Bayesian Trie is called valid, if all of its nodes are valid for the trie's global current time, (N_Z, M_Z).*

Definition 2.29 (Node Correctness)**.** *A node is "correct" if it correctly calculates the unnormalized posterior with respect to its local time,*

$$x.Z = Z_{x.n_Z, x.m_Z}(x)$$

Definition 2.30 (Bayesian Trie Correctness)**.** *The Bayesian trie is "correct" if all of its nodes are correct.*

We may now give the algorithm to advance a node's time. This algorithm recalculates the unnormalized posterior of a valid node, x, to a new local time, (n'_Z, m'_Z).

Algorithm 1 Advance Node Time

Require: x, the node to update.
Require: $(x.n_Z, x.m_Z)$, the node's current time.
Require: (n'_Z, m'_Z), the new time.
Require: (N_Z, M_Z), the trie's current time.
Require: x is valid at time (N_Z, M_Z).
Require: $n'_Z \geq x.n_Z$ and $m'_Z \geq x.m_Z$.
1: $x.Z \leftarrow x.Z * (P^{br}_{m'_Z}(x) / P^{br}_{x.m_Z}(x)) * \prod_{i=x.n_Z+1}^{n'_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$
2: $x.n_Z \leftarrow n'_Z$
3: $x.m_Z \leftarrow m'_Z$

We also provide for advancing the trie to a new global time.

Algorithm 2 Advance Trie Time

Require: Bayesian Trie B is valid at its current time (N_Z, M_Z).
Require: (N'_Z, M'_Z), the new global time.
1: $B.N_Z \leftarrow N'_Z$
2: $B.M_Z \leftarrow M'_Z$
3: $F \leftarrow \bigvee_{i=1}^{N'_Z} C_i^L \wedge \left(\bigvee_{i=N_Z+1}^{N'_Z} C_i^L \vee \bigvee_{i=M_Z+1}^{M'_Z} C_i^P \right)$ ▷ $x \geq F$ implies x is valid at time (N'_Z, M'_Z).
4: Run `AdvanceNodeTime` to update the time of each node in this frontier to (N'_Z, M'_Z).
5: **for** each node x strictly within F, $x < F$, proceeding in reverse topological order **do**
6: $\quad x.Z \leftarrow \sum_{\lambda \in \Lambda} (x + \lambda).\mathrm{Z}$
7: $\quad x.n_Z \leftarrow N'_Z$
8: $\quad x.m_Z \leftarrow M'_Z$

We now show that both algorithms preserve the validity of the trie and its correctness.

Theorem 2.1 (Advance Node Time Validity)**.** *The `AdvanceNodeTime` algorithm preserves the validity of the node.*

Proof. The "constant likelihood" condition depends only on the global time, not the node's local time, so updating the node's time does not affect whether this condition is satisfied. The "no reweighting" condition is weakened by increasing the node's time. I.e.,

$$x \geq \bigvee_{i=x.n_Z+1}^{N_Z} C_i^L \implies x \geq \bigvee_{i=n'_Z+1}^{N_Z} C_i^L$$

and,

$$x \geq \bigvee_{i=x.m_Z+1}^{M_Z} C_i^P \implies x \geq \bigvee_{i=m'_Z+1}^{M_Z} C_i^P$$

if $n'_Z \geq x.n_Z$ and $m'_Z \geq x.m_Z$. □

Lemma 2.5 (Prior Delta Factorization)**.** *Let $m \leq m'$ and let x be an at most properly terminated string satisfying $x \geq \bigvee_{i=m+1}^{m'} C_i^P$. Then*

$$P^{br}_{m'}(x) = P^{br}_{m}(x) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}).$$

Proof. By the definition of a prior delta digest sequence,

$$P_{m'}(h) = P_m(h) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor h \rfloor_{C_i^P}) \qquad \forall h \in H.$$

Since $x \geq \bigvee_{i=m+1}^{m'} C_i^P$, the same argument as in lemma 2.2 shows that for every $h \geq x$ and every $i \in \{m+1, \ldots, m'\}$,

$$\lfloor h \rfloor_{C_i^P} = \lfloor x \rfloor_{C_i^P}.$$

Hence, for all $h \geq x$,

$$P_{m'}(h) = P_m(h) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}).$$

Summing over all properly terminated descendants of x gives

$$\begin{aligned} P_{m'}^{br}(x) &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) \\ &= \sum_{\substack{h \in H \\ h \geq x}} P_m(h) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) \\ &= \left(\sum_{\substack{h \in H \\ h \geq x}} P_m(h) \right) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) \\ &= P_m^{br}(x) \prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}), \end{aligned}$$

as claimed. □

Corollary 2.1 (Prior Delta Quotient)**.** *Under the hypotheses of lemma 2.5, if $P_m^{br}(x) > 0$, then*

$$\prod_{i=m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) = \frac{P_{m'}^{br}(x)}{P_m^{br}(x)}.$$

Theorem 2.2 (Advance Node Time Correctness)**.** *The* `AdvanceNodeTime` *algorithm preserves the correctness of the node.*

Proof. By correctness before the update,

$$x.Z = Z_{x.n_Z, x.m_Z}(x).$$

We show that line **??** changes this value to $Z_{n'_Z, m'_Z}(x)$ in each of the two validity cases.

Constant Likelihood Case: Suppose $x \geq \bigvee_{i=1}^{N_Z} C_i^L$. Applying lemma 2.2 with $f = x$ gives

$$\begin{aligned} x.Z &= Z_{x.n_Z, x.m_Z}(x) \\ &= \sum_{\substack{h \in H \\ h \geq x}} P_{x.m_Z}(h) L_{x.n_Z}(h) \\ &= \sum_{\substack{h \in H \\ h \geq x}} P_{x.m_Z}(h) \prod_{i=1}^{x.n_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\ &= P^{br}_{x.m_Z}(x) \prod_{i=1}^{x.n_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}). \end{aligned}$$

Therefore, after line **??**,

$$\begin{aligned} x.Z &= P^{br}_{x.m_Z}(x) \prod_{i=1}^{x.n_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \left(\frac{P^{br}_{m'_Z}(x)}{P^{br}_{x.m_Z}(x)} \right) \prod_{i=x.n_Z+1}^{n'_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\ &= P^{br}_{m'_Z}(x) \prod_{i=1}^{n'_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\ &= \left(\sum_{\substack{h \in H \\ h \geq x}} P_{m'_Z}(h) \right) \prod_{i=1}^{n'_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\ &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'_Z}(h) L_{n'_Z}(h) \\ &= Z_{n'_Z, m'_Z}(x), \end{aligned}$$

where the third equality is by definition 2.17, the fourth is by lemma 2.2, and the final equality is by definition of $Z_{n'_Z, m'_Z}$.

No Reweighting Case: Suppose $x \geq \bigvee_{i=x.n_Z+1}^{N_Z} C_i^L$ and $x \geq \bigvee_{i=x.m_Z+1}^{M_Z} C_i^P$. For every $h \in H$ with $h \geq x$, the same truncation argument as in lemma 2.2 gives

$$L_{n'_Z}(h) = L_{x.n_Z}(h) \prod_{i=x.n_Z+1}^{n'_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}),$$

$$P_{m'_Z}(h) = P_{x.m_Z}(h) \prod_{i=x.m_Z+1}^{m'_Z} \Delta_i^P(\lfloor x \rfloor_{C_i^P}).$$

Hence

$$
\begin{aligned}
Z_{n'_Z,m'_Z}(x) &= \sum_{\substack{h\in H\\ h\geq x}} P_{m'_Z}(h) L_{n'_Z}(h) \\
&= \sum_{\substack{h\in H\\ h\geq x}} P_{x.m_Z}(h) L_{x.n_Z}(h) \prod_{i=x.m_Z+1}^{m'_Z} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) \prod_{i=x.n_Z+1}^{n'_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\
&= Z_{x.n_Z,x.m_Z}(x) \prod_{i=x.m_Z+1}^{m'_Z} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) \prod_{i=x.n_Z+1}^{n'_Z} \Delta_i^L(\lfloor x \rfloor_{C_i^L}).
\end{aligned}
$$

By corollary 2.1,

$$
\prod_{i=x.m_Z+1}^{m'_Z} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) = \frac{P^{br}_{m'_Z}(x)}{P^{br}_{x.m_Z}(x)}.
$$

Substituting this and using $x.Z = Z_{x.n_Z,x.m_Z}(x)$ shows that line **??** sets $x.Z$ to $Z_{n'_Z,m'_Z}(x)$. □

Lemma 2.6 (Frontier Calculation)**.** *If $F = \bigvee_{i=1}^{N'_Z} C_i^L \wedge \left(\bigvee_{i=N_Z+1}^{N'_Z} C_i^L \vee \bigvee_{i=M_Z+1}^{M'_Z} C_i^P\right)$, $x \geq F$, and x is valid at time (N_Z, M_Z), then x is valid at time (N'_Z, M'_Z).*

Proof. If $x \geq F$, then either $x \geq \bigvee_{i=1}^{N'_Z} C_i^L$ or both $x \geq \bigvee_{i=N_Z+1}^{N'_Z} C_i^L$ and $x \geq \bigvee_{i=M_Z+1}^{M'_Z} C_i^P$. The first case is exactly the "Constant Likelihood" branch in the definition of validity at time (N'_Z, M'_Z). So we only need to consider the second case.

The other assumption, that x is valid at time (N_Z, M_Z), also has two cases: either $x \geq \bigvee_{i=1}^{N_Z} C_i^L$ or $x \geq \bigvee_{i=x.n_Z+1}^{N_Z} C_i^L$ and $x \geq \bigvee_{i=x.m_Z+1}^{M_Z} C_i^P$.

Constant Likelihood for time (N_Z, M_Z) **Case:** Suppose x is valid at time (N_Z, M_Z) because it satisfies the constant likelihood condition, i.e., $x \geq \bigvee_{i=1}^{N_Z} C_i^L$. We may combine this with our previous result that $x \geq \bigvee_{i=N_Z+1}^{N'_Z} C_i^L$, in order to write, $x \geq \bigvee_{i=1}^{N'_Z} C_i^L$, which is the "Constant Likelihood" branch for validity at time (N'_Z, M'_Z).

No Reweighting Case for time (N_Z, M_Z) **Case:** Suppose x is valid at time (N_Z, M_Z) because it satisfies the no reweighting condition, i.e., $x \geq \bigvee_{i=x.n_Z+2}^{N_Z} C_i^L$ and $x \geq \bigvee_{i=x.m_Z+1}^{M_Z} C_i^P$. We may combine these with our previous result that $x \geq \bigvee_{i=N_Z+1}^{N'_Z} C_i^L$ and $x \geq \bigvee_{i=M_Z+1}^{M'_Z} C_i^P$, in order to write, $x \geq \bigvee_{i=x.n_Z+1}^{N'_Z} C_i^L$ and $x \geq \bigvee_{i=x.m_Z+1}^{M'_Z} C_i^P$, which is the "No Reweighting" condition for validity at time (N'_Z, M'_Z). □

Theorem 2.3 (Advance Trie Time Validity)**.** *We must show that after running* `AdvanceTrieTime`*, the trie is valid at its new time, (N'_Z, M'_Z).*

Proof. If $x > F$, then x is unaffected by the algorithm and is valid at the new time by lemma 2.6. Otherwise, $x \leq F$. All such nodes are updated to have the new time $(x.n_Z, x.m_Z) = (N'_Z, M'_Z)$. By lemma 2.4, these nodes are valid at their new time. □

Theorem 2.4 (Advance Trie Time Correctness)**.** *The* `AdvanceTrieTime` *algorithm preserves the correctness of the trie.*

Proof. We show this by induction on the trie.

We are concerned with nodes, $x \leq F$, since nodes which are beyond the update boundary, $x > F$, are unaffected by the algorithm.

At the start of the algorithm, all nodes are valid and correct.

Base Case: If $x \in F$, the x is valid and correct and we have already shown that running `AdvanceNodeTime` on x preserves its validity and correctness.

Induction Step: If $x < F$, then, because we process nodes in reverse topological order, by the induction hypothesis, all its immediate children are correct and have time (N', M'). Thus line 6 recalculates the unnormalized posterior of x correctly by lemma 2.3. □

2.2 Likelihood Tracking

The preceding algorithm is correct, but we wish to avoid keeping all the Δ_i^L in memory globally and the cost of calculating truncations, $\lfloor x \rfloor_{C_i^L}$. To do so we introduce a field called "unpushed likelihood", $x.l$, which will be used to track the product of likelihood deltas as well as a corresponding local "likelihood tracking time", $x.n_t$. We also augment our Bayesian Trie with a global "likelihood tracking time", $B.N_t$.

(Formally, these fields are independent of the global and local likelihood times of the used in the preceding section, altough when we put the algorithms together, they will be the same.)

We define our invariant, "push correctness":

Definition 2.31 (Push Correctness)**.** *We say that a Bayesian trie B is push correct for time N_t if, every node has local likelihood tracking time at most N_t, and, for every node whose descendant likelihoods have changed by a constant factor, that factor is equal to the product of the "unpushed likelihood" fields of all of its ancestors (including itself). I.e.,*

$$\forall x,\ x.n_t \leq N_t$$

and

$$\forall x,\ x \geq \bigvee_{i=x.n_t+1}^{N_t} C_i^L \implies \prod_{y \leq x} y.l = \prod_{i=x.n_t+1}^{N_t} \Delta_i^L\big(\lfloor x \rfloor_{C_i^L}\big)$$

This is invariant under two very simple algorithms:

Algorithm 3 Push Likelihood

Require: Bayesian Trie B, Node x
Require: B is push correct for time $B.N_t$.
Require: $\forall a < x, a.l = 1$ ▷ x does not have ancestors with unpushed likelihood
1: $y.l \leftarrow y.l * x.l \quad \forall y \in x.\text{children}$
2: $x.l \leftarrow 1$
3: $x.n_t \leftarrow B.N_t$

Algorithm 4 Apply Likelihood Delta

Require: Bayesian Trie B, Node x
Require: B is push correct for time $B.N_t$.
1: **for** each node $x \in C_{B.N_t+1}^L$ **do**
2: $\quad x.l \leftarrow x.l * \Delta_{B.N_t+1}^L(x)$
3: $B.N_t \leftarrow B.N_t + 1$

Theorem 2.5 (Push Likelihood Correctness)**.** *Algorithm 3 preserves the push correctness of the trie.*

Proof. Let $N = B.N_t$, and let B' be the trie after the algorithm is applied. We show that the defining equality of push correctness still holds for every node z.

First consider $z \neq x$. If $z \not> x$, then none of the ancestors of z are modified by the algorithm, so $\prod_{y \leq z} y.l$ is unchanged. If $z > x$, then the algorithm removes the factor $x.l$ from x and multiplies it into each child of x. Along the ancestor chain of z, exactly one child of x occurs, so the product $\prod_{y \leq z} y.l$ is again unchanged. In either subcase, $z.n_t$ is unchanged and $B'.N_t = N$, so the right-hand side of Definition 2.31 is unchanged as well. Since B was push correct before the algorithm, the equality continues to hold for z in B'.

Now consider $z = x$. After the algorithm, $x.l = 1$ and $x.n_t = N$. By the precondition, every strict ancestor $a < x$ satisfies $a.l = 1$. Therefore

$$\prod_{y \leq x} y.l = 1.$$

On the other hand, because $x.n_t = B'.N_t = N$, the product on the right-hand side of Definition 2.31 is an empty product, hence also equal to 1. Thus the equality holds for x as well.

Therefore B' is push correct for time N. □

Theorem 2.6 (Apply Likelihood Delta Correctness)**.** *Algorithm 4 preserves the push correctness of the trie.*

Proof. Let N be the value of $B.N_t$ before the algorithm, and let B' be the trie after the algorithm. Then $B'.N_t = N + 1$.

Fix a node z such that

$$z \geq \bigvee_{i=z.n_t+1}^{N+1} C_i^L.$$

We must show that

$$\prod_{y \leq z} y.l = \prod_{i=z.n_t+1}^{N+1} \Delta_i^L\big(\lfloor z \rfloor_{C_i^L}\big)$$

holds in B'.

Since $z \geq \bigvee_{i=z.n_t+1}^{N+1} C_i^L$, in particular $z \geq C_{N+1}^L$. Hence $\lfloor z \rfloor_{C_{N+1}^L}$ is defined and is an ancestor of z. Because C_{N+1}^L is a prefix code, this is the unique node of C_{N+1}^L on the ancestor chain of z.

The algorithm multiplies $u.l$ by $\Delta_{N+1}^L(u)$ for each $u \in C_{N+1}^L$. Therefore, along the ancestor chain of z, exactly one factor is introduced, namely

$$\Delta_{N+1}^L\big(\lfloor z \rfloor_{C_{N+1}^L}\big).$$

So in B' we have

$$\prod_{y \leq z} y.l = \left(\prod_{y \leq z} y.l\right)_B \Delta_{N+1}^L\big(\lfloor z \rfloor_{C_{N+1}^L}\big).$$

Since B was push correct for time N,

$$\left(\prod_{y \leq z} y.l\right)_B = \prod_{i=z.n_t+1}^{N} \Delta_i^L\big(\lfloor z \rfloor_{C_i^L}\big).$$

Substituting gives

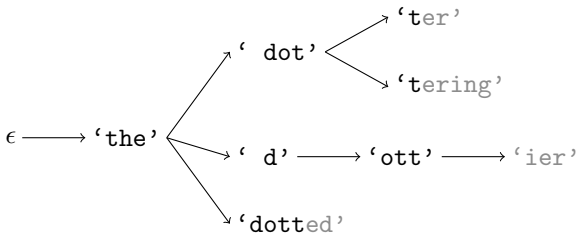
$$\prod_{y \leq z} y.l = \left(\prod_{i=z.n_t+1}^{N} \Delta_i^L\big(\lfloor z \rfloor_{C_i^L}\big)\right) \Delta_{N+1}^L\big(\lfloor z \rfloor_{C_{N+1}^L}\big) = \prod_{i=z.n_t+1}^{N+1} \Delta_i^L\big(\lfloor z \rfloor_{C_i^L}\big).$$

This is exactly the required equality, so B' is push correct for time $N + 1$. □

2.3 Character Priors from Token Models

Since Dotter uses a letter-by-letter trie, but language models return conditional token probabilities, we must convert a token model to a literal model. A given literal prefix corresponds to several token paths. For example, the prefix '`the dott`' could be part of the phrases with the following tokenizations (using the GPT-2 tokenizer):

- `the• dot•ter` (three tokens),
- `the• dot•tering` (three tokens),
- `the• d•ott•ier` (four tokens),
- `the• dotted` (two tokens),



We can see that a literal prefix, '`the dott`', can correspond to multiple token prefixes, such as [`'the'`, `' dot'`], [`'the'`, `' d'`], and [`'the'`]. For each token prefix, the literal prefix can be generated from multiple possible terminal tokens. For example, given the token prefix [`'the'`, `' dot'`], the literal prefix is compatible with either of the terminal tokens '`ter`' or '`tering`'.

In this section, we formalize tokenization, introduce a key property of byte-pair encoders called boundary sufficiency, and prove theorem 2.7 which allows us to compute the prior of a literal prefix by summing over all token sequences that pass through it.

Terminology Note: It is helpful to make three kinds of distinctions: (1) between sets and their elements, (2) between sequences (either strings or token sequences) and their elements, and (3) between characters and tokens. We will choose to use uppercase letters for sets, Greek letters for individual characters or tokens, and bold font for token sequences. Table 2.1 demonstrates these conventions.

Example Symbol	Type
l	string
L	set of strings
λ	character
Λ	alphabet
$\boldsymbol{l}$	token sequence
$\boldsymbol{L}$	set of token sequences
$\boldsymbol{\lambda}$	token
$\boldsymbol{\Lambda}$	vocabulary (set of tokens)

Table 2.1: Example Symbols and Their Types

Definition 2.32 (Vocabulary)**.** *A vocabulary is a finite set. We will use the notation* $\boldsymbol{\Lambda}$ *to represent our vocabulary.*

Definition 2.33 (Token)**.** *A token is an element of a vocabulary.*

Definition 2.34 (Token Sequence)**.** *A token sequence is a sequence of tokens.*

Throughout this chapter we will use the notation $\boldsymbol{\Lambda}$ to refer to our vocabulary. We will require that our vocabulary contain a special "stop" token, denoted, $\boldsymbol{\omega}$. The set of all token sequences will be denoted $\boldsymbol{S}$.

Definition 2.35 (Properly Terminated Token Sequence)**.** *A token sequence is properly terminated if and only if it ends with the stop token, with all preceding tokens not being the stop token. I.e.,*

$$\forall i \quad (i = |\boldsymbol{a}| - 1 \iff \boldsymbol{a}[i] = \boldsymbol{\omega})$$

Analogously to the string case, we will use the notation $\boldsymbol{H}$ to represent the set of all properly terminated token sequences.

Definition 2.36 (At Most Properly Terminated Token Sequence)**.** *A token sequence,* $\boldsymbol{a} \in \boldsymbol{S}$*, is at most properly terminated if and only if it is either properly terminated or contains no stop token, i.e.,* $\boldsymbol{a} \in \boldsymbol{H}$ *or* $\nexists i\ \boldsymbol{a}[i] = \boldsymbol{\omega}$*.*

We will use the symbol $\mathbf{A}$ to represent the set of all admissible token sequences.

Definition 2.37 (Spelling Function)**.** *The spelling function,* ρ*, maps a token to a string*

$$\rho : \boldsymbol{\Lambda} \to \boldsymbol{S}$$

we require that,

$$\rho(\boldsymbol{\omega}) = \omega$$

Definition 2.38 (Literalization Function)**.** *The literalization function,* T^{-1}*, maps a token sequence to a string by concatenating their spellings. That is,*

$$T^{-1}(\mathbf{a}) = \rho(\mathbf{a}[0])\ \rho(\mathbf{a}[1])\ \cdots\ \rho(\mathbf{a}[-1])$$

where juxtaposition denotes string concatenation.

Definition 2.39 (Tokenization Function)**.** *A tokenization function is a mapping from strings to token sequences,* $T : S \to \mathbf{S}$ *that is inverted by literalization, i.e.,*

$$T^{-1}(T(x)) = x \quad \forall x \in S$$

(but the converse will not usually hold, i.e., different token sequences may correspond to the same string.) We also require that the tokenization function maps properly terminated strings to properly terminated token sequences,

$$T : H \to \mathbf{H}$$

so in particular the final stop character of a properly terminated string tokenizes as the stop token.

Definition 2.40 (Canonical Token Sequence)**.** *A token sequence* $\mathbf{a}$ *is canonical if and only if re-tokenizing its literalization yields the same token sequence:*

$$T(T^{-1}(\mathbf{a})) = \mathbf{a}.$$

We will use the symbol $\mathbf{C}$ to represent the set of all canonical token sequences.

Proposition 2.3. *For every string* x*, the token sequence* $T(x)$ *is canonical.*

Proof. By definition,

$$T(T^{-1}(T(x))) = T(x).$$

□

Definition 2.41 (Admissible Token Sequence)**.** *A token sequence is admissible if and only if it is canonical and at most properly terminated.*

Definition 2.42 (String versus Token Sequence Comparison)**.** *Since literalization is a well defined function, we compare token sequences and strings, by first literalizing the token sequence to a string. For example we can write,* $\mathbf{a} \leq x$*, to mean,* $T^{-1}(\mathbf{a}) \leq x$*. It does not mean that* $\mathbf{a} \leq T(x)$*.*

Definition 2.43 (Token Sequence String Subtraction)**.** *We can perform subtraction between a string and a token sequence by first literalizing the token sequence to a string and then applying definition 2.7.*

Definition 2.44 (Token Sequence versus Complete Prefix Code Comparison)**.** *We also compare token sequences and complete prefix codes by first literalizing the token sequence to a string. We write* $\mathbf{a} \leq C$ *to mean* $T^{-1}(\mathbf{a}) \leq C$*, and* $\mathbf{a} > C$ *to mean* $T^{-1}(\mathbf{a}) > C$*.*

Definition 2.45 (Token Sequence versus Token Sequence Comparison)**.** *We compare token sequences as sequences of symbols, not as literalized strings. That is,* $\mathbf{a} \leq \mathbf{b}$ *means that* $\exists i, \mathbf{a} = \mathbf{b}[:i]$*, it does* not *mean that* $T^{-1}(\mathbf{a}) \leq T^{-1}(\mathbf{b})$*.*

Definition 2.46 (Boundary Sufficiency)**.** *The boundary sufficiency property of a tokenization function states that every contiguous subsequence of a canonical token sequence remains canonical when retokenized in isolation. Formally, a tokenization function* T *has boundary sufficiency if for every canonical token sequence* $\mathbf{a} \in \mathbf{C}$ *and every* $0 \leq i \leq j \leq |\mathbf{a}|$*,*

$$T(T^{-1}(\mathbf{a}[i:j])) = \mathbf{a}[i:j].$$

Henceforth, we will always assume that our tokenization function has boundary sufficiency. This is because all byte-pair encoders have this property.

Proposition 2.4 (Token-sequence prefixes of a tokenization are canonical)**.** *If* $\mathbf{b} \leq T(x)$*, then* $\mathbf{b} \in \mathbf{C}$*.*

Proof. Since $T(x) \in \mathbf{C}$ by proposition 2.3 and $\mathbf{b} \leq T(x)$, there exists $j \leq |T(x)|$ such that $\mathbf{b} = T(x)[:j]$. Applying boundary sufficiency to the canonical token sequence $T(x)$ with indices 0 and j gives

$$T(T^{-1}(\mathbf{b})) = T(T^{-1}(T(x)[:j])) = T(x)[:j] = \mathbf{b},$$

so $\mathbf{b} \in \mathbf{C}$. □

We can now formalize a token-based language model.

Definition 2.47 (Conditional Token Prior Function)**.** *Our conditional token prior function tells us the conditional probability of a token given a token sequence. It is a mapping*

$$\mathbf{Q} : \mathbf{\Lambda}^* \times \mathbf{\Lambda} \to \mathbb{R}^+$$

written as $\mathbf{Q}(\boldsymbol{\tau} \mid \mathbf{a})$*, where,*

$$\forall \mathbf{a} \in \mathbf{\Lambda}^*, \sum_{\boldsymbol{\tau} \in \mathbf{\Lambda}} \mathbf{Q}(\boldsymbol{\tau} \mid \mathbf{a}) = 1.$$

We also require canonical support:

$$\mathbf{Q}(\boldsymbol{\tau} \mid \mathbf{a}) = 0 \qquad \textit{whenever } \mathbf{a} \in \mathbf{A} \textit{ and } \mathbf{a} + \boldsymbol{\tau} \notin \mathbf{A}.$$

We also require almost-sure termination: for every admissible token sequence $\mathbf{a} \in \mathbf{A} \setminus \mathbf{H}$*,*

$$\sum_{\mathbf{s} \in \mathbf{H}} \prod_{i < |\mathbf{s}|} \mathbf{Q}(\mathbf{s}[i] \mid \mathbf{a} + \mathbf{s}[:i]) = 1.$$

Explanatory Note: The almost-sure termination condition is an assumption on the conditional token prior, not a lemma. Local normalization of $\mathbf{Q}(\boldsymbol{\tau} \mid \mathbf{a})$ only guarantees that the next token is sampled from a probability distribution; by itself it does not imply that the process eventually emits the stop token with probability 1.

Definition 2.48 (Conditional Token Prior Function Sequence)**.** *A conditional token prior function sequence is a sequence of conditional token prior functions,* $\{\mathbf{Q}_n\}$*.*

Definition 2.49 (Token Prior Function)**.** *A token prior function is a mapping from properly terminated token sequences to non-negative real numbers,*

$$\mathbf{P}_m : \mathbf{H} \to \mathbb{R}^+,$$

satisfying

$$\sum_{\mathbf{h} \in \mathbf{H}} \mathbf{P}_m(\mathbf{h}) = 1.$$

Definition 2.50 (Token Branch Prior Function)**.** *The token branch prior of a token sequence* $\mathbf{a}$ *is*

$$\mathbf{P}_m^{br}(\mathbf{a}) := \sum_{\substack{\mathbf{h} \in \mathbf{H} \\ \mathbf{h} \geq \mathbf{a}}} \mathbf{P}_m(\mathbf{h}).$$

Definition 2.51 (Induced Token Prior)**.** *Given a conditional token prior function* $\mathbf{Q}_m$*, we define the induced token prior on properly terminated token sequences by*

$$\mathbf{P}_m(\mathbf{h}) := \prod_{i < |\mathbf{h}|} \mathbf{Q}_m(\mathbf{h}[i] \mid \mathbf{h}[:i]).$$

Proposition 2.5 (Induced Token Prior Is a Token Prior)**.** *The induced token prior is a token prior function.*

Proof. By definition, $\mathbf{P}_m : \mathbf{H} \to \mathbb{R}^+$. It remains to show that

$$\sum_{\mathbf{h} \in \mathbf{H}} \mathbf{P}_m(\mathbf{h}) = 1.$$

Since the empty token sequence is admissible and not properly terminated, the almost-sure termination assumption gives

$$\sum_{\mathbf{h} \in \mathbf{H}} \prod_{i < |\mathbf{h}|} \mathbf{Q}_m(\mathbf{h}[i] \mid \mathbf{h}[:i]) = 1.$$

By definition 2.51, the summand is exactly $\mathbf{P}_m(\mathbf{h})$, so the claim follows. □

Lemma 2.7 (Token Branch Prior Product Formula)**.** *For every admissible token sequence* $\mathbf{a} \in \mathbf{A}$*,*

$$\mathbf{P}_m^{br}(\mathbf{a}) = \prod_{i<|\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]).$$

Proof. We split into two cases.

If $\mathbf{a} \in \mathbf{H}$, then $\mathbf{a}$ is already properly terminated. No properly terminated token sequence can be a strict extension of $\mathbf{a}$, since that would place the stop token before the final index. Therefore $\mathbf{a}$ is the unique properly terminated descendant of itself, so

$$\mathbf{P}_m^{br}(\mathbf{a}) = \mathbf{P}_m(\mathbf{a}).$$

By definition 2.51, this is exactly

$$\prod_{i<|\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]).$$

Now suppose $\mathbf{a} \in \mathbf{A} \setminus \mathbf{H}$. Every properly terminated token sequence $\mathbf{h} \geq \mathbf{a}$ can be written uniquely as $\mathbf{h} = \mathbf{a} + \mathbf{s}$ where $\mathbf{s} \in \mathbf{H}$. Therefore,

$$\begin{aligned}
\mathbf{P}_m^{br}(\mathbf{a}) &= \sum_{\substack{\mathbf{h}\in\mathbf{H} \\ \mathbf{h}\geq\mathbf{a}}} \mathbf{P}_m(\mathbf{h}) \\
&= \sum_{\mathbf{s}\in\mathbf{H}} \mathbf{P}_m(\mathbf{a}+\mathbf{s}) \\
&= \sum_{\mathbf{s}\in\mathbf{H}} \left(\prod_{i<|\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]) \right) \left(\prod_{i<|\mathbf{s}|} \mathbf{Q}_m(\mathbf{s}[i] \mid \mathbf{a}+\mathbf{s}[:i]) \right) \\
&= \left(\prod_{i<|\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]) \right) \sum_{\mathbf{s}\in\mathbf{H}} \prod_{i<|\mathbf{s}|} \mathbf{Q}_m(\mathbf{s}[i] \mid \mathbf{a}+\mathbf{s}[:i]) \\
&= \prod_{i<|\mathbf{a}|} \mathbf{Q}_m(\mathbf{a}[i] \mid \mathbf{a}[:i]),
\end{aligned}$$

where the final step is the almost-sure termination assumption in definition 2.47. □

Henceforth we will study prior functions that are derived from token priors induced from conditional token priors by the relation,

$$P_n(h) = \mathbf{P}_n(T(h)).$$

We can now state our first result

Theorem 2.7 (Token Prior Path Summation)**.** *The branch prior of a string* a*, is the sum of the token branch priors of all token sequences that pass through* a*.*

$$P^{br}(a) = \begin{cases} 1, & a = \epsilon, \\ \displaystyle\sum_{\substack{\mathbf{b}\in\mathbf{A} \\ \mathbf{b}<a}} \mathbf{P}^{br}(\mathbf{b}) \sum_{\substack{\boldsymbol{\tau}\in\boldsymbol{\Lambda} \\ a\leq\mathbf{b}+\boldsymbol{\tau}}} \mathbf{Q}(\boldsymbol{\tau} \mid \mathbf{b}), & a \neq \epsilon. \end{cases}$$

Proof. We prove the two cases separately.

If $a = \epsilon$, then by Definition 2.17,

$$P^{br}(\epsilon) = \sum_{\substack{h\in H \\ h\geq\epsilon}} P(h) = \sum_{h\in H} P(h) = 1.$$

Now let $a \neq \epsilon$. Again from Definition 2.17,

$$P^{br}(a) = \sum_{\substack{h \in H \\ h \geq a}} P(h).$$

For each $h \geq a$, write $\mathbf{c} := T(h)$. Since $\mathbf{c} \geq a$, there exists a unique smallest j such that $a \leq \mathbf{c}[:j+1]$. Define $\mathbf{b} := \mathbf{c}[:j]$ and $\boldsymbol{\tau} := \mathbf{c}[j]$. By minimality of j, we have

$$\mathbf{b} < a \leq \mathbf{b} + \boldsymbol{\tau}.$$

So each $h \geq a$ determines a unique pair $(\mathbf{b}, \boldsymbol{\tau})$ with those inequalities.

Grouping the sum over h by these pairs gives

$$P^{br}(a) = \sum_{\substack{\mathbf{b} \in \mathbf{A} \\ \mathbf{b} < a}} \sum_{\substack{\boldsymbol{\tau} \in \boldsymbol{\Lambda} \\ a \leq \mathbf{b} + \boldsymbol{\tau}}} \sum_{\substack{h \in H: \\ (\mathbf{b}+\boldsymbol{\tau}) \leq T(h)}} P(h).$$

By the defining relation $P_m(h) = \mathbf{P}_m(T(h))$, the innermost sum is

$$\sum_{\substack{\mathbf{h} \in \mathbf{H} \\ \mathbf{h} \geq \mathbf{b} + \boldsymbol{\tau}}} \mathbf{P}_m(\mathbf{h}),$$

which is exactly $\mathbf{P}^{br}(\mathbf{b} + \boldsymbol{\tau})$ by definition 2.50. By lemma 2.7,

$$\mathbf{P}^{br}(\mathbf{b} + \boldsymbol{\tau}) = \mathbf{P}^{br}(\mathbf{b})\, \mathbf{Q}(\boldsymbol{\tau} \mid \mathbf{b}).$$

Substituting yields

$$P^{br}(a) = \sum_{\mathbf{b} \in \mathbf{A},\, \mathbf{b} < a} \mathbf{P}^{br}(\mathbf{b}) \sum_{\boldsymbol{\tau} \in \boldsymbol{\Lambda},\, a \leq \mathbf{b} + \boldsymbol{\tau}} \mathbf{Q}(\boldsymbol{\tau} \mid \mathbf{b}),$$

as claimed. □

2.4 Prior Tracking

We now provide an algorithm to implement theorem 2.7. Thus it suffices to compute $P^{br}_{M'_P}(x)$ on the valid frontier. We now formalize a finite dynamic program that does exactly this.

Definition 2.52 (Token Fan)**.** *Let m be a prior time, let $\mathbf{b} \in \mathbf{A}$ be an admissible token sequence, and let x be an at most properly terminated string. The token fan of x from context $\mathbf{b}$ at time m is*

$$\Phi_m(\mathbf{b}, x) := \sum_{\boldsymbol{\tau} \in \boldsymbol{\Lambda},\, x \leq \mathbf{b} + \boldsymbol{\tau}} \mathbf{Q}_m(\boldsymbol{\tau} \mid \mathbf{b}).$$

Definition 2.53 (Token Ancestor String)**.** *For a non-empty string x, we define its token ancestor string by*

$$\pi(x) := T^{-1}(T(x)[:-1]).$$

Definition 2.54 (String-Prefix Context)**.** *For a string x and a character index $i \leq |x|$, we define the string-prefix context*

$$\kappa_x(i) := T(x[:i]).$$

(and for convenience, we allow this notation for negative indices too).

Explanatory Note: The sum for a branch prior runs over *string* ancestors of x, not merely over token ancestors of x in its own tokenization. For each strict string prefix $x[:i]$, we re-tokenize that shorter string to obtain the context $\kappa_x(i)$. In general, $\kappa_x(i)$ need not be a prefix of $T(x)$. This is exactly why algorithm 6 must sum over all strict string ancestors.

Lemma 2.8 (Token Prefix Retokenization)**.** *For every string x and every $j \le |T(x)|$,*

$$T(T^{-1}(T(x)[:j])) = T(x)[:j].$$

Proof. Since $T(x) \in \mathbf{C}$, the claim follows immediately from boundary sufficiency applied to the canonical token sequence $T(x)$ with indices 0 and j. □

Lemma 2.9 (Nonadmissible Token Branch Priors Vanish)**.** *If a token sequence $\mathbf{a}$ is not admissible, then*

$$\mathbf{P}_m^{br}(\mathbf{a}) = 0.$$

Proof. If $\mathbf{a}$ is not at most properly terminated, then no properly terminated token sequence can extend $\mathbf{a}$, since that would preserve an internal stop token. Hence the index set in definition 2.50 is empty, so $\mathbf{P}_m^{br}(\mathbf{a}) = 0$.

Otherwise, $\mathbf{a}$ is at most properly terminated but not canonical. Let j be the least index such that $\mathbf{a}[:j{+}1]$ is not canonical. Then $\mathbf{a}[:j]$ is canonical, and because $\mathbf{a}$ is at most properly terminated, the sequence $\mathbf{a}[:j{+}1]$ is not admissible. By admissible support,

$$\mathbf{Q}_m(\mathbf{a}[j] \mid \mathbf{a}[:j]) = 0.$$

Every properly terminated descendant $\mathbf{h} \ge \mathbf{a}$ has the same prefix $\mathbf{a}[:j{+}1]$, so its induced token prior contains the same zero factor. Therefore $\mathbf{P}_m(\mathbf{h}) = 0$ for every such descendant, and hence

$$\mathbf{P}_m^{br}(\mathbf{a}) = 0.$$

□

Theorem 2.8 (String-Ancestor Reindexing)**.** *For every non-empty at most properly terminated string x,*

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(\kappa_x(i))\, \Phi_m(\kappa_x(i), x).$$

Proof. By theorem 2.7,

$$P_m^{br}(x) = \sum_{\mathbf{b} \in \mathbf{A},\, \mathbf{b} < x} \mathbf{P}_m^{br}(\mathbf{b})\, \Phi_m(\mathbf{b}, x).$$

We now reindex this sum by the literal prefix

$$s := T^{-1}(\mathbf{b}).$$

Since $\mathbf{b} \in \mathbf{A}$, in particular $\mathbf{b}$ is canonical, so

$$\mathbf{b} = T(T^{-1}(\mathbf{b})) = T(s),$$

so $\mathbf{b} = \kappa_x(|s|)$. Since $s < x$, this index satisfies $|s| < |x|$.

Thus the admissible contexts are in one-to-one correspondence with the strict string prefixes $x[:i]$ for $0 \le i < |x|$, via $\mathbf{b} = \kappa_x(i) = T(x[:i])$. Substituting this reindexing into the path-summation formula yields

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(\kappa_x(i))\, \Phi_m(\kappa_x(i), x),$$

as claimed. □

We provide two short algorithms to compute the priors. The essential thing is to know the "token branch prior", written $x.\mathbf{P}$, of the ancestors of a node before computing its branch prior, $x.P$. We augment nodes with times tracking when these calculations are valid, $x.m_P$ and $x.m_{\mathbf{P}}$. Correspondingly, we use global times M_P and $M_{\mathbf{P}}$ for the trie. In the present algorithms these are always updated in sync, so $M_P = M_{\mathbf{P}}$, but it is clearer to distinguish them notationally.

Unlike our likelihood tracking, and our core algorithm, which both lazily reuse calculations across iterations, this section requires us to rerun the prior calculations from the root on each update. This makes the presentation simple.

Algorithm 5 Compute Token Branch Prior

Require: Bayesian Trie B, Node x, New Time m
Require: Every node in B stores correct prior values for its recorded times.
Require: $x = \epsilon$ or $\pi(x).m_{\mathbf{P}} = m$ ▷ The token ancestor of x already has the new time.

if $x = \epsilon$ **then**
    $x.\mathbf{P} \leftarrow 1$
else
    $x.\mathbf{P} \leftarrow \pi(x).\mathbf{P} * \mathbf{Q}_m(T(x)[-1] \mid T(x)[:-1])$
$x.m_{\mathbf{P}} \leftarrow m$

Algorithm 6 Compute Branch Prior

Require: Bayesian Trie B, Node x, New Time m
Require: Every node in B stores correct prior values for its recorded times.
Require: $(x[:i]).m_{\mathbf{P}} = m \quad \forall i < |x|$ ▷ All string ancestors of x already have calculated the token branch prior at the new time.

$x.P \leftarrow \sum_{i=0}^{|x|-1}(x[:i]).\mathbf{P} * \Phi_m(\kappa_x(i), x)$ ▷ Theorem 2.8.
$x.m_P \leftarrow m$

We now show the correctness of these two algorithms.

Theorem 2.9 (Compute Token Branch Prior Correctness)**.** *Algorithm 5 correctly computes the token branch prior of a node.*

Proof. We must show that after the algorithm runs,

$$x.\mathbf{P} = \mathbf{P}_m^{br}(T(x)).$$

There are two cases.

If $x = \epsilon$, the algorithm sets $x.\mathbf{P} \leftarrow 1$. Also $T(\epsilon) = \epsilon$, so it suffices to show that

$$\mathbf{P}_m^{br}(\epsilon) = 1.$$

By definition 2.50,

$$\mathbf{P}_m^{br}(\epsilon) = \sum_{\mathbf{h} \in \mathbf{H}} \mathbf{P}_m(\mathbf{h}),$$

which is equal to 1 because $\mathbf{P}_m$ is a token prior function. Hence

$$\mathbf{P}_m^{br}(T(x)) = \mathbf{P}_m^{br}(\epsilon) = 1 = x.\mathbf{P},$$

so the stored value is correct.

Now suppose $x \neq \epsilon$. By the precondition, the token ancestor $\pi(x)$ already has time m, so its stored value is correct:

$$\pi(x).\mathbf{P} = \mathbf{P}_m^{br}(T(\pi(x))).$$

By lemma 2.8,

$$T(\pi(x)) = T(x)[:-1].$$

Thus,

$$\pi(x).\mathbf{P} = \mathbf{P}_m^{br}(T(x)[:-1]).$$

The algorithm then sets

$$x.\mathbf{P} \leftarrow \pi(x).\mathbf{P} \cdot \mathbf{Q}_m(T(x)[-1] \mid T(x)[:-1]).$$

Substituting the known ancestor value gives

$$x.\mathbf{P} = \mathbf{P}_m^{br}(T(x)[:-1]) \cdot \mathbf{Q}_m(T(x)[-1] \mid T(x)[:-1]).$$

By lemma 2.7,

$$\mathbf{P}_m^{br}(T(x)) = \mathbf{P}_m^{br}(T(x)[:-1]) \cdot \mathbf{Q}_m(T(x)[-1] \mid T(x)[:-1]).$$

Therefore $x.\mathbf{P} = \mathbf{P}_m^{br}(T(x))$, as required.

Finally, the assignment $x.m_{\mathbf{P}} \leftarrow m$ records exactly the time at which this value is valid. □

Theorem 2.10 (Compute Branch Prior Correctness)**.** *Algorithm 6 correctly computes the branch prior of a node.*

Proof. We must show that after the algorithm runs,

$$x.P = P_m^{br}(x).$$

By the precondition, every strict string ancestor $x[:i]$ has already had its token branch prior computed at time m. Thus for every $i < |x|$,

$$(x[:i]).\mathbf{P} = \mathbf{P}_m^{br}(T(x[:i])).$$

By theorem 2.8,

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(\kappa_x(i))\, \Phi_m(\kappa_x(i), x).$$

Using definition 2.54, this becomes

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} \mathbf{P}_m^{br}(T(x[:i]))\, \Phi_m(T(x[:i]), x).$$

Replacing each $\mathbf{P}_m^{br}(T(x[:i]))$ by the already-computed stored value $(x[:i]).\mathbf{P}$ gives

$$P_m^{br}(x) = \sum_{i=0}^{|x|-1} (x[:i]).\mathbf{P}\, \Phi_m(\kappa_x(i), x).$$

This is exactly the value assigned to $x.P$ by the algorithm. Therefore the algorithm correctly computes the branch prior of x.

Finally, the assignment $x.m_P \leftarrow m$ records the time at which this branch prior value is valid. □

Two implementation remarks are useful in practice:

1. The relevant contexts for a literal prefix x are governed only by token boundaries that can occur within the final token overlapping x. When tokens cannot contain spaces internally, this means that only prefixes of the final word need be considered. For a tokenizer with maximum token length L, this limits the search to at most the last L characters.
2. If the vocabulary is stored in lexicographic order and the model output is converted to cumulative probabilities, then the fan $\Phi_M(\mathbf{b}, x)$ can be obtained from a difference of cumulative sums over the contiguous lexicographic range of tokens $\boldsymbol{\tau}$ satisfying $x \leq T^{-1}(\mathbf{b} + \boldsymbol{\tau})$. (Given how we defined our comparison conventions it is permissible to write this as, $\boldsymbol{\tau} \leq \mathbf{b} - x$.)

2.5 Maximum Truncation Compatible Descendant Likelihood

We now discuss how to track the maximum truncation compatible descendant likelihood for nodes. The quantity is not needed to compute posteriors and is orthogonal to the preceding discussion. However, it will be helpful later for determining which token sequences should be evaluated by the language model.

The algorithm somewhat resembles the core posterior calculation algorithm, in that we begin by updating the computation on a frontier and then update it within the frontier with a kind of up-propagation algorithm.

We begin by defining truncating a string to a token sequence.

Definition 2.55 (Token Truncation)**.** *The token truncation of a string x under a token sequence $\mathbf{a}$ is the longest prefix of $\mathbf{a}$ that is also, as a string, a prefix of x. It maps a string to a token sequence, and is written as $\lfloor x \rfloor_{\mathbf{a}}$. (Context makes it clear when this is intended as opposed to complete prefix code truncation, which uses the same notation, but with a complete prefix code instead of a token sequence in the subscript.)*

$$\lfloor x \rfloor_{\mathbf{a}} = \max_{\substack{\mathbf{b} \leq \mathbf{a} \\ \mathbf{b} \leq x}} \mathbf{b}$$

Lemma 2.10 (Token truncation is transitive along string prefixes)**.** *If $x \leq f$, then for every token sequence $\mathbf{c}$,*

$$\lfloor x \rfloor_{\lfloor f \rfloor_{\mathbf{c}}} = \lfloor x \rfloor_{\mathbf{c}}.$$

Proof. By definition,

$$\lfloor x \rfloor_{\lfloor f \rfloor_{\mathbf{c}}} = \max_{\substack{\mathbf{b} \leq \lfloor f \rfloor_{\mathbf{c}} \\ \mathbf{b} \leq x}} \mathbf{b}.$$

Since

$$\lfloor f \rfloor_{\mathbf{c}} = \max_{\substack{\mathbf{d} \leq \mathbf{c} \\ \mathbf{d} \leq f}} \mathbf{d},$$

a token sequence $\mathbf{b}$ satisfies $\mathbf{b} \leq \lfloor f \rfloor_{\mathbf{c}}$ if and only if $\mathbf{b} \leq \mathbf{c}$ and $\mathbf{b} \leq f$.

Therefore the indexing set above is exactly

$$\{\mathbf{b} \mid \mathbf{b} \leq \mathbf{c},\ \mathbf{b} \leq f,\ \mathbf{b} \leq x\}.$$

Because $x \leq f$, every token sequence whose string is a prefix of x is automatically a prefix of f. Hence this set is equal to

$$\{\mathbf{b} \mid \mathbf{b} \leq \mathbf{c},\ \mathbf{b} \leq x\}.$$

Taking the maximum over this set gives

$$\lfloor x \rfloor_{\lfloor f \rfloor_{\mathbf{c}}} = \max_{\substack{\mathbf{b} \leq \mathbf{c} \\ \mathbf{b} \leq x}} \mathbf{b} = \lfloor x \rfloor_{\mathbf{c}},$$

as claimed. □

Definition 2.56 (Maximum Truncation Compatible Descendant Likelihood)**.** *The maximum truncation compatible descendant likelihood is the maximum likelihood of any descendant, h, for which the canonical tokenization is an extension of $\mathbf{a}$ and makes $\mathbf{a}$ the token truncation of x. If there is no such descendant, this maximum is taken to be 0.*

$$R_n(\mathbf{a}, x) = \max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} L_n(h)$$

In the special case where the string corresponds to the token sequence, this reduces to,

Lemma 2.11 (Identical Token and String Prefixes)**.** *If $\mathbf{a} = T(a)$, then*

$$R_n(\mathbf{a}, a) = \max_{\substack{h \in H \\ \mathbf{a} \leq T(h)}} L_n(h)$$

Proof. If $\mathbf{a}$ is properly terminated then $h = a$ satisfies the conditions. Otherwise, if $\mathbf{a}$ is not terminated, then $h = a + \boldsymbol{\omega}$ satisfies the conditions, since the stop character is always required to tokenize as its own token. □

This latter quantity, the maximum likelihood of any descendant of a token sequence, is the quantity of interest, but we will need a datastructure that maintains $R_n(\mathbf{a}, x)$ for other values of x, $x > \mathbf{a}$.

Lemma 2.12 (Frontier ancestors of a descendant extending a covered prefix)**.** *Let F be a prefix code, let x be a string satisfying $x \le F$, and let h be a string satisfying $x \le h$. If $f \in F$ and $f \le h$, then $x \le f$.*

Proof. Because $x \le F$, there exists some $g \in F$ such that $x \le g$. Since both f and x are prefixes of h, they are comparable. If $f < x$, then $f < x \le g$, so f and g are comparable distinct elements of the prefix code F, a contradiction. Hence $f \not< x$, and therefore $x \le f$. □

The core theorem required for our algorithm relates the truncation compatible descendant likelihood to the same quantity on an arbitrary frontier.

Theorem 2.11 (Truncation Compatible Descendant Likelihood from the Frontier)**.** *Given a canonical token sequence $\mathbf{a}$, a string x, a complete prefix code $F \le H$, and a likelihood time n, such that $x \le F$,*

$$R_n(\mathbf{a}, x) = \max_{\substack{f \in F \\ x \le f}} \max_{\substack{\mathbf{b} \in \mathbf{C} \\ \mathbf{a} \le \mathbf{b} \le f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f),$$

Proof. We prove both inequalities.

First, let $h \in H$ be any string satisfying

$$x \le h, \quad \mathbf{a} \le T(h), \quad \lfloor x \rfloor_{T(h)} = \mathbf{a}.$$

Because $F \le H$ and $h \in H$, there exists $f \in F$ such that $f \le h$. By lemma 2.12, $x \le f$, so $x \le f \le h$.

Let

$$\mathbf{b} := \lfloor f \rfloor_{T(h)}.$$

Then $\mathbf{b} \le T(h)$, so proposition 2.4 gives $\mathbf{b} \in \mathbf{C}$. Also $\mathbf{b} \le f$ and $\mathbf{a} \le \mathbf{b}$. Moreover, by lemma 2.10,

$$\lfloor x \rfloor_{\mathbf{b}} = \lfloor x \rfloor_{\lfloor f \rfloor_{T(h)}} = \lfloor x \rfloor_{T(h)} = \mathbf{a}.$$

Since $f \le h$, $\mathbf{b} \le T(h)$, and $\lfloor f \rfloor_{T(h)} = \mathbf{b}$, the string h is admissible in the definition of $R_n(\mathbf{b}, f)$. Hence

$$L_n(h) \le R_n(\mathbf{b}, f).$$

Because every admissible h for $R_n(\mathbf{a}, x)$ yields such a pair $(\mathbf{b}, f)$, this proves

$$R_n(\mathbf{a}, x) \le \max_{\substack{f \in F \\ x \le f}} \max_{\substack{\mathbf{b} \in \mathbf{C} \\ \mathbf{a} \le \mathbf{b} \le f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f).$$

For the reverse inequality, let $\mathbf{b} \in \mathbf{C}$ and $f \in F$ satisfy

$$x \le f, \quad \mathbf{a} \le \mathbf{b} \le f, \quad \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}.$$

If there is no $h \in H$ admissible in the definition of $R_n(\mathbf{b}, f)$, then $R_n(\mathbf{b}, f) = 0 \le R_n(\mathbf{a}, x)$ by definition. Otherwise, let $h \in H$ satisfy

$$f \le h, \quad \mathbf{b} \le T(h), \quad \lfloor f \rfloor_{T(h)} = \mathbf{b}.$$

Since $x \le f \le h$, we have $x \le h$. Also, again by lemma 2.10,

$$\lfloor x \rfloor_{T(h)} = \lfloor x \rfloor_{\lfloor f \rfloor_{T(h)}} = \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}.$$

Because $\mathbf{a} \le \mathbf{b} \le T(h)$, the string h is admissible in the definition of $R_n(\mathbf{a}, x)$. Hence every value contributing to $R_n(\mathbf{b}, f)$ also contributes to $R_n(\mathbf{a}, x)$, so

$$R_n(\mathbf{b}, f) \le R_n(\mathbf{a}, x).$$

Taking the maximum over all such $\mathbf{b}$ and f gives

$$\max_{\substack{f \in F \\ x \le f}} \max_{\substack{\mathbf{b} \in \mathbf{C} \\ \mathbf{a} \le \mathbf{b} \le f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f) \le R_n(\mathbf{a}, x).$$

Therefore the two sides are equal. □

Lemma 2.13 (Canonical token prefixes of a string)**.** *Let f be a string. The canonical token sequences $\mathbf{b}$ satisfying $\mathbf{b} \leq f$ are exactly those of the form*

$$\mathbf{b} = T(f[:i])$$

for some $i \leq |f|$.

Proof. First suppose $\mathbf{b} \leq f$ and $\mathbf{b}$ is canonical. Then there exists a string $b = T^{-1}(\mathbf{b})$ with $b \leq f$. Writing $i := |b|$, we have $b = f[:i]$, and therefore

$$\mathbf{b} = T(b) = T(f[:i]).$$

Conversely, let $i \leq |f|$. Then $f[:i] \leq f$, so

$$T^{-1}(T(f[:i])) = f[:i] \leq f.$$

Thus $T(f[:i])$ is canonical and satisfies $T(f[:i]) \leq f$. □

Corollary 2.2 (Truncation Compatible Descendant Likelihood from the Frontier, Reindexed)**.** *Given a canonical token sequence $\mathbf{a}$, a string x, a complete prefix code F, and a likelihood time n, such that $x \leq F$ and $H \geq F$,*

$$R_n(\mathbf{a}, x) = \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{i \leq |f| \\ \lfloor x \rfloor_{T(f[:i])} = \mathbf{a}}} R_n(T(f[:i]), f).$$

Proof. By theorem 2.11,

$$R_n(\mathbf{a}, x) = \max_{\substack{f \in F \\ x \leq f}} \max_{\substack{\mathbf{b} \in \mathbf{C} \\ \mathbf{a} \leq \mathbf{b} \leq f \\ \lfloor x \rfloor_{\mathbf{b}} = \mathbf{a}}} R_n(\mathbf{b}, f).$$

For fixed $f \in F$, lemma 2.13 shows that the canonical token sequences $\mathbf{b}$ satisfying $\mathbf{b} \leq f$ are exactly those of the form $\mathbf{b} = T(f[:i])$ for $i \leq |f|$. Reindexing the outer maximum by this correspondence yields the claim. □

This prepares us for our first, algorithm,

We compute a quanitity, $\widehat{R}(\mathbf{a}, x)$. In implementation, this is data attached to the node, x, indexed by its ancestors, $\mathbf{a}$. Our goal is to design an algorithm that correctly computes $\widehat{R}(\mathbf{a}, x)$ at time N_R, i.e., $\widehat{R}(\mathbf{a}, x) = R_{N_R}(\mathbf{a}, x)$.

Algorithm 7 MTCDL Up Propagation

Require: Complete Prefix Code, F
Require: $H \geq F$
Require: New max likelihood tracking time, N_R
Require: $x.n_R = N_R$ ▷ All frontier nodes already have the new max likelihood tracking time
Require: $f \in F \implies \widehat{R}(\mathbf{d}, f) = R_{N_R}(\mathbf{d}, f)$ for all canonical $\mathbf{d} \leq f$. ▷ MTCDLFA Already Correct at Frontier
Require: $x < F \implies \widehat{R}(\mathbf{d}, x) \leq R_{N_R}(\mathbf{d}, x)$ for all canonical $\mathbf{d} \leq x$.

```
for each node f ∈ F do
    for i ≤ |f| do
        b ← T(f[:i])                          ▷ Canonical tokenizations of each string ancestor of f
        for j from |f| down to 0 do
            R̂(⌊f[:j]⌋_b, f[:j]) ←_max R̂(b, f)
for each node x < F do                        ▷ Update the local max likelihood tracking time of all nodes
    x.n_R ← N_R
```

Theorem 2.12 (MTCDL Up Propagation Correctness)**.** *Under the hypotheses of algorithm 7, if $H \geq F$, $\mathbf{a} \leq F$, and $x \leq F$, then*

$$R_{N_R}(\mathbf{a}, x) = \widehat{R}(\mathbf{a}, x).$$

Proof. Fix $\mathbf{a} \le F$ and $x \le F$. By inspection of algorithm 7, the value $\widehat{R}(\mathbf{a}, x)$ at termination is the maximum of its initial value and all values $\widehat{R}(T(f[:i]), f)$ such that $f \in F$, $x \le f$, and $\lfloor x \rfloor_{T(f[:i])} = \mathbf{a}$. Thus

$$\widehat{R}(\mathbf{a}, x) = \max\left(\widehat{R}^{\text{init}}(\mathbf{a}, x),\ \max_{\substack{f \in F \\ x \le f}} \max_{\substack{i \le |f| \\ \lfloor x \rfloor_{T(f[:i])} = \mathbf{a}}} \widehat{R}(T(f[:i]), f)\right).$$

By the initialization hypothesis, $\widehat{R}^{\text{init}}(\mathbf{a}, x) \le R_{N_R}(\mathbf{a}, x)$. Also, for every $f \in F$ and $i \le |f|$, the frontier hypothesis gives

$$\widehat{R}(T(f[:i]), f) = R_{N_R}(T(f[:i]), f).$$

Therefore

$$\widehat{R}(\mathbf{a}, x) = \max\left(\widehat{R}^{\text{init}}(\mathbf{a}, x),\ \max_{\substack{f \in F \\ x \le f}} \max_{\substack{i \le |f| \\ \lfloor x \rfloor_{T(f[:i])} = \mathbf{a}}} R_{N_R}(T(f[:i]), f)\right).$$

By the reindexed corollary above, the second term is exactly $R_{N_R}(\mathbf{a}, x)$. Hence the maximum above is $\max(\widehat{R}_{\text{init}}(\mathbf{a}, x), R_{N_R}(\mathbf{a}, x)) = R_{N_R}(\mathbf{a}, x)$. □

The preceding algorithm explains how to update within the frontier to a new likelihood time. We now give a simple update algorithm for beyond the frontier.

Algorithm 8 MTCDL Update

Require: Node x
Require: New max likelihood tracking time n'_R
Require: $n'_R \ge x.n_R$ ▷ The new likelihood time is not earlier than the node's local time
Require: $\widehat{R}(\mathbf{d}, x) = R_{x.n_R}(\mathbf{d}, x)$ for all canonical $\mathbf{d} \le x$.
Require: $x \ge \bigvee_{i=x.n_R+1}^{n'_R} C_i^L$
1: $\widehat{R}(\mathbf{d}, x) * = \prod_{i=x.n_R+1}^{n'_R} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$ for all canonical $\mathbf{d} \le x$
2: $x.n_R \leftarrow n'_R$

Lemma 2.14 (MTCDL Likelihood Factorization)**.** *Let $n \le n'$ and let x be a string satisfying*

$$x \ge \bigvee_{i=n+1}^{n'} C_i^L.$$

Then for every canonical token sequence $\mathbf{a} \le x$,

$$R_{n'}(\mathbf{a}, x) = R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}).$$

Proof. By the definition of a likelihood delta digest sequence, for every $h \in H$,

$$L_{n'}(h) = L_n(h) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}).$$

Since $x \ge \bigvee_{i=n+1}^{n'} C_i^L$, the same argument as in lemma 2.2 shows that for every $h \ge x$ and every $i \in \{n+1, \ldots, n'\}$,

$$\lfloor h \rfloor_{C_i^L} = \lfloor x \rfloor_{C_i^L}.$$

Hence, for all $h \ge x$,

$$L_{n'}(h) = L_n(h) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}).$$

If there is no $h \in H$ admissible in the definition of $R_n(\mathbf{a}, x)$, then both $R_n(\mathbf{a}, x)$ and $R_{n'}(\mathbf{a}, x)$ are 0 by definition, and the claim follows. Otherwise,

$$\begin{aligned}
R_{n'}(\mathbf{a}, x) &= \max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} L_{n'}(h) \\
&= \max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} \left(L_n(h) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \right) \\
&= \left(\max_{\substack{h \in H \\ x \leq h \\ \mathbf{a} \leq T(h) \\ \lfloor x \rfloor_{T(h)} = \mathbf{a}}} L_n(h) \right) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\
&= R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}),
\end{aligned}$$

as claimed. □

Theorem 2.13 (MTCDL Update Correctness)**.** *Under the hypotheses of algorithm 8, after execution,*

$$\widehat{R}(\mathbf{a}, x) = R_{n'}(\mathbf{a}, x)$$

for all canonical $\mathbf{a} \leq x$*.*

Proof. Let $n := x.n_R$ before execution of the algorithm, and fix a canonical token sequence $\mathbf{a} \leq x$. By the precondition,

$$\widehat{R}(\mathbf{a}, x) = R_n(\mathbf{a}, x).$$

By lemma 2.14,

$$R_{n'}(\mathbf{a}, x) = R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}).$$

The algorithm multiplies $\widehat{R}(\mathbf{a}, x)$ by this same factor, so the updated value satisfies

$$\begin{aligned}
\widehat{R}(\mathbf{a}, x) &= R_n(\mathbf{a}, x) \prod_{i=n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \\
&= R_{n'}(\mathbf{a}, x).
\end{aligned}$$

Since $\mathbf{a}$ was arbitrary, the conclusion holds for all canonical $\mathbf{a} \leq x$. □

We now discuss machinery to initialize the maximum truncation compatible descendant likelihood for nodes at time $N_R = 0$. For this, we need the further assumption that our tokenizer has the property of canonical concatenation.

Definition 2.57 (Canonical Concatenation)**.** *A tokenizer has the property of canonical concatenation if overlapping canonical token sequences—with the overlap nonempty—can be concatenated to form a longer canonical token sequence. I.e.,*

$$\textit{For all } \mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbf{C}, \ \ \mathbf{b} \neq \epsilon,$$

$$\Big(\mathbf{a} + \mathbf{b} \in \mathbf{C} \ \wedge \ \mathbf{b} + \mathbf{c} \in \mathbf{C} \Big) \implies \mathbf{a} + \mathbf{b} + \mathbf{c} \in \mathbf{C}$$

In section 3.1, we show that any BPE tokenizer has the property of canonical concatenation.

Definition 2.58 (Possible Truncation Predicate)**.** *The predicate, $\phi(\mathbf{a}, x)$, indicates whether there exists a descendant h of x whose tokenization truncates x to $\mathbf{a}$.*

$$\phi(\mathbf{a}, x) \iff \exists h \left(x \leq h \wedge \lfloor x \rfloor_{T(h)} = \mathbf{a} \right)$$

Definition 2.59 (Canonical Token Pair Predicate)**.** *The canonical token pair predicate, $\psi(\boldsymbol{\tau_1}, \boldsymbol{\tau_2})$, indicates whether a pair of tokens are canonical.*

$$\psi(\boldsymbol{\tau_1}, \boldsymbol{\tau_2}) \iff T(\rho(\boldsymbol{\tau_1}) + \rho(\boldsymbol{\tau_2})) = [\boldsymbol{\tau_1}, \boldsymbol{\tau_2}]$$

Definition 2.60 (String Vocabulary)**.** *The set of strings corresponding to the vocabulary is the "string vocabulary", denoted V.*

Definition 2.61 (Vocabulary Prefix Set)**.** *The vocabulary prefix set, $V_{\leq}$, is the set of all strings that are prefixes of some string in the vocabulary, V.*

Definition 2.62 (Canonical Token Prefix Pair Predicate)**.** *We define the canonical token prefix pair predicate, ξ to indicate whether a given prefix could canonically follow a given token.*

$$\xi : \boldsymbol{\Lambda} \times V_{\leq} \to \mathbb{B}$$

with

$$\xi(\boldsymbol{\tau_1}, v) \iff \exists \boldsymbol{\tau_2} \in \boldsymbol{\Lambda}, \psi(\boldsymbol{\tau_1}, \boldsymbol{\tau_2}) \wedge v \leq \boldsymbol{\tau_2}$$

Corollary 2.3 (BPE Last-Token Extension)**.** *Assume the tokenization function is induced by a BPE merge system as in section 3.1. Let $\mathbf{a} \in \mathbf{C}$ be nonempty, and let $\boldsymbol{\tau}$ be a token. If*

$$\psi(\mathbf{a}[-1], \boldsymbol{\tau}),$$

then

$$\mathbf{a} + \boldsymbol{\tau} \in \mathbf{C}.$$

Proof. This is exactly corollary 3.1. □

Theorem 2.14 (Possible Truncation Predicate from Canonical Token Prefix Pair Predicate)**.** *We may calculate the possible truncation predicate, ϕ, from the canonical token prefix pair predicate, ξ.*

$$\phi(\mathbf{a}, x) \iff (\mathbf{a} = \epsilon \wedge x \in V_{\leq}) \vee \xi(\mathbf{a}[-1], x - \mathbf{a})$$

Proof. We prove both directions.

If $\phi(\mathbf{a}, x)$ is true, then there exists a descendant h of x whose tokenization truncates x to $\mathbf{a}$. If $\mathbf{a} = \epsilon$, then $x \leq \rho(T(h)[0])$, so $x \in V_{\leq}$ and we are done. Otherwise, let $\boldsymbol{\tau} = \mathbf{a}[-1]$, and $\boldsymbol{\tau_2} = T(h)[|\mathbf{a}|]$, the next token in $T(h)$ after the last token of $\mathbf{a}$. We have $x - \mathbf{a} \leq \boldsymbol{\tau_2}$ and by the boundary sufficiency of $T(h)$ we have $\psi(\boldsymbol{\tau}, \boldsymbol{\tau_2})$, hence $\xi(\boldsymbol{\tau}, x - \mathbf{a})$ holds.

Conversely, if $\xi(\boldsymbol{\tau}, x - \mathbf{a})$ is true, then there exists a token $\boldsymbol{\tau_2}$ such that $\psi(\boldsymbol{\tau}, \boldsymbol{\tau_2}) \wedge (x - \mathbf{a} \leq \boldsymbol{\tau_2})$. We may then construct $h = T^{-1}(\mathbf{a} + \boldsymbol{\tau_2} + \boldsymbol{\omega})$ which by corollary 2.3 has $T(h) = \mathbf{a} + \boldsymbol{\tau_2} + \boldsymbol{\omega}$, guaranteeing that $x \leq h$ and $\lfloor x \rfloor_{T(h)} = \mathbf{a}$.

□

Theorem 2.15 (Maximum Truncation Compatible Descendant Likelihood from Possible Truncation Predicate)**.** *Recall that $\forall h,\ L_0(h) = 1$. Then*

$$R_0(\mathbf{a}, x) = \begin{cases} 1 & \text{if } \phi(\mathbf{a}, x), \\ 0 & \text{otherwise.} \end{cases}$$

Proof. □

We can now give our initialization algorithm.

Algorithm 9 Initialize MTCDL

Require: Bayesian Trie B
Require: Node x
Require: Tokenizer with property of canonical concatenation.

```
x.n_R ← 0
for i ≤ |x| do
    a ← T(x[: i])
    if (i = 0 ∧ x[: i] ∈ V_≤) or ξ(a[−1], x[: i]) then
        R̂(a, x) ← 1
    else
        R̂(a, x) ← 0
```

Efficient Evaluation of the Possible Truncation Predicate

Corollary 2.3 shows that, in the BPE setting, the question of whether a candidate next token preserves canonicality depends only on the last token of the current canonical token sequence. This motivates the predicate ξ.

I do not currently have a complete exact characterization of the possible truncation predicate, ϕ, solely in terms of ξ and a vocabulary-prefix lookup.

Computing the predicate $\in V_{\leq}$ is straightforward. One solution would be to maintain a trie of the string vocabulary, V.

The former predicate, ξ, is much more difficult, and has resisted my attempts to find an efficient algorithm. However, it can be solved in $O(1)$ time by precomputing a lookup table for all possible pairs in $\mathbf{\Lambda} \times V_{\leq}$. Using reasonable values of $|\mathbf{\Lambda}| = 50,000$ and $|V_{\leq}| = 100,000$, this table has 5,000,000,000 entries. Assuming 1 bit of memory per entry, this is about 600MB.

I studied the problem of compressing the closely related lookup table for the canonical token pair predicate, ψ. I found that for a random choice of $\boldsymbol{\sigma}$ and $\boldsymbol{\tau}$, the probability that $\boldsymbol{\sigma} + \boldsymbol{\tau}$ tokenizes to $[\boldsymbol{\sigma}, \boldsymbol{\tau}]$ is about 1/2. Knowing either argument does not help very much: the conditional entropy is still about 0.6 bits in either case.

However, since this predicate is only required on the server and not in the browser, we can afford a 600MB lookup table, since the language model will be several gigabytes anyway.

An Upper-Bound Problem for Possible Truncations

In the implementation, each node stores a finite boolean array `truncation_possible`, indexed by candidate token-ancestor depths. For a fixed node, the number of true entries measures how many different token-ancestor choices remain compatible with the current string prefix. This quantity is important because it controls the storage cost of several per-node arrays.

We therefore seek an upper bound on

$$\max_x \sum_i \mathbf{1}\big[\phi_i(x)\big],$$

where $\phi_i(x)$ denotes the event that the ith token-ancestor candidate for x is possible. The naive upper bound obtained from ξ alone is too weak, because it allows each suffix position to choose an unrelated witness token independently.

To make the dependence on the string explicit, let $T \in V$ be a token and let $p \in V_{\leq}$ be a vocabulary prefix that does not begin with the tokenizer's distinguished word-start marker, and define

$$x = T + p.$$

Equivalently, among all vocabulary prefixes we only consider those that do not introduce a fresh word boundary inside x. We then consider suffix boundaries j in x, and write

$$p' = x[j :].$$

In the loosest version of the bound, one counts such a j whenever there exists an $i < j$ such that

$$\tau = x[i:j] \in V \qquad \text{and} \qquad \xi(\tau, p').$$

This is indeed an upper bound, but it is weak because different values of j may be certified by completely unrelated token witnesses τ.

The goal is therefore to strengthen the legality condition on j by forcing the witness token to arise from a coherent tokenization of the substring ending at j.

Token-suffix notation. We define the token-suffix set

$$V_{\text{suffix}} = \{\, s \mid \exists \tau \in V,\ s \leq_{\text{suffix}} \tau \,\},$$

that is, the set of all suffixes of strings in the vocabulary.

Definition 2.63 (Canonical token-suffix pair predicate)**.** *We define*

$$\zeta : V_{\text{suffix}} \times \mathbf{\Lambda} \to \mathbb{B}$$

by

$$\zeta(s, \tau) \iff \exists \sigma \in \mathbf{\Lambda},\ s \leq_{\text{suffix}} \sigma \wedge \psi(\sigma, \tau).$$

Thus $\zeta(s, \tau)$ records whether the token τ can be canonically preceded by some token whose suffix is s.

Stricter legality condition. Fix $x = T + p$ and a boundary j, and let $p' = x[j:]$. Rather than allowing an arbitrary token witness $\tau = x[i:j]$, we impose the following stronger condition:

There exists an index $h \leq j$ such that if

$$\mathbf{m} = T(x[h:j]),$$

then all of the following hold:

$$\mathbf{m}[-1] = \tau,$$

$$\xi(\tau, p') \text{ holds},$$

$$\zeta(x[:h], \mathbf{m}[0]) \text{ holds}.$$

Here one chooses h so that the middle tokenization $T(x[h:j])$ is as long as possible, i.e. h is the start of the first complete token lying entirely inside $x[:j]$. The string $x[:h]$ is then interpreted only through its final token suffix, which is exactly the domain captured by V_{suffix} and the predicate ζ.

Concrete tokenization example. The witness patterns above are easy to see in the `tikzpicture` family. (This is for the tiny-llama tokenizer, which agrees exactly with "https://tiktokenizer.vercel.app/?model=codellama%2FCodeL 7b-hf".) Under the filtered TinyLlama tokenizer, the relevant exact string tokenizations are

$$T(\texttt{tikzpicturoid}) = [\texttt{tikz}, \texttt{p}, \texttt{ict}, \texttt{u}, \texttt{roid}],$$

$$T(\texttt{tikzpictur}) = [\texttt{tikz}, \texttt{p}, \texttt{ict}, \texttt{ur}],$$

$$T(\texttt{tikzpicturn}) = [\texttt{tikz}, \texttt{pic}, \texttt{turn}],$$

$$T(\texttt{tikzpictures}) = [\texttt{tikz}, \texttt{p}, \texttt{ictures}],$$

$$T(\texttt{tikzpicture}) = [\texttt{tikzpicture}],$$

$$T(\texttt{atikzpicture}) = [\texttt{at}, \texttt{ikz}, \texttt{picture}].$$

These provide concrete witnesses for why a prefix such as `tikzpictur` can support many different legal suffix boundaries j: the right-hand witness token may be `roid`, `ur`, `turn`, `ictures`, or `tikzpicture`, while the left-hand side can also contribute a nontrivial suffix witness, as in the tokenization of `atikzpicture`.

Resulting search procedure. The proposed upper-bound algorithm is therefore:

Algorithm 10 A Stricter Upper Bound on the Number of Possible Truncations

```
best ← 0
for T ∈ V do
    for p ∈ V_≤ with p not beginning with the word-start marker do
        x ← T + p
        count ← 0
        for boundaries j of x do
            p′ ← x[j :]
            if p′ ∉ V_≤ then
                continue
            legal ← false
            for indices h ≤ j do
                m ← T(x[h : j])
                if m is empty then
                    continue
                τ ← m[−1]
                if ξ(τ, p′) ∧ ζ(x[: h], m[0]) then
                    legal ← true
                    break
            if legal then
                count ← count + 1
        best ← max(best, count)
return best
```

Status. This stronger procedure is intended to remove the obvious weakness of the independent-j upper bound by forcing each legal witness to arise from a real tokenization bridge inside x. At present, however, I do not have a proof that this condition is sufficient to characterize the true maximum number of possible truncations, nor a proof that it is the tightest efficiently computable upper bound.

Running this algorithm on the filtered tiny-llama tokenizer gives the following histograms:

Yes. Rewritten as percentages:

For $\max_T \text{count}(T, p)$ over prefixes p ($N_{\text{prefix}} = 12{,}554$):

- $1 \to 6$ / 12,554 = **0.048**%
- $2 \to 2{,}397$ / 12,554 = **19.092**%
- $3 \to 7{,}371$ / 12,554 = **58.716**%
- $4 \to 2{,}553$ / 12,554 = **20.336**%
- $5 \to 219$ / 12,554 = **1.745**%
- $6 \to 8$ / 12,554 = **0.064**%

For all scanned (T, p) pairs ($N_{\text{pairs}} = 216{,}368{,}190$):

- $0 \to 2{,}487$ / 216,368,190 = **0.001**%
- $1 \to 13{,}871{,}959$ / 216,368,190 = **6.411**%
- $2 \to 116{,}901{,}205$ / 216,368,190 = **54.029**%
- $3 \to 73{,}838{,}841$ / 216,368,190 = **34.126**%
- $4 \to 11{,}420{,}636$ / 216,368,190 = **5.278**%
- $5 \to 332{,}311$ / 216,368,190 = **0.154**%
- $6 \to 751$ / 216,368,190 = **0.00035**%

I believe that every instance of six possible truncations is not actually realizable since they would require inconsistent preceding characters, but I do not have a computationally feasible way to prove this. Remember

that the previous algorithm was only an upper bound. However, the witness provided at the start of this subsection is a concrete example of five possible truncations. The utility of this is that it lets us see that an array of [f32; 6] and a bitmap of possible truncations will suffice here.

2.6 The Complete Algorithms

Here we combine the preceding sections to give Dotter's complete algorithms. The tracking of the maximum descendant likelihood is optional and will only run on the server. We denote the trie's global maximum descendant likelihood tracking time by $B.N_R$, corresponding to the local times $x.n_R$.

Algorithm 11 Complete Prior Update

Require: Bayesian Trie B.

$N'_Z \leftarrow B.N_Z$
$M'_Z \leftarrow B.M_Z + 1$
$M'_P \leftarrow B.M_P + 1$
$M'_{\mathbf{P}} \leftarrow B.M_{\mathbf{P}} + 1$ ▷ In the present algorithms, M'_Z, M'_P, and $M'_{\mathbf{P}}$ remain synchronized.
$F \leftarrow \emptyset$
for each node x in the trie in topological order **do**
▷ Run algorithm 5, Compute Token Branch Prior, on x with new time $M'_{\mathbf{P}}$.
    if $x = \epsilon$ **then**
        $x.\mathbf{P} \leftarrow 1$
    else
        $x.\mathbf{P} \leftarrow \pi(x).\mathbf{P} * \mathbf{Q}_{M'_{\mathbf{P}}}(T(x)[-1] \mid T(x)[: -1])$
    $x.m_{\mathbf{P}} \leftarrow M'_{\mathbf{P}}$
    ▷ Run algorithm 3, Push Likelihood, on x.
    $y.l \leftarrow y.l * x.l \quad \forall y \in x.\text{children}$
    $x.l \leftarrow 1$
    $x.n_t \leftarrow B.N_t$
    ▷ Locate the update frontier, F.
    if $x \geq \bigvee_{i=1}^{N'_Z} C_i^L$ or $x \geq C^P_{M'_Z}$ **then**
        $F \leftarrow F \cup \{x\}$
        Break descent from x
for each node $x \in F$ **do**
▷ Run algorithm 6, Compute Branch Prior, on x with new time M'_P.
    $x.P \leftarrow \sum_{i=0}^{|x|-1}(x[: i]).\mathbf{P} * \Phi_{M'_P}(\kappa_x(i), x)$
    $x.m_P \leftarrow M'_P$
▷ Run the first half of algorithm 2, Advance Trie Time, on B to (N'_Z, M'_Z).
▷ Run algorithm 1, Advance Node Time, on x to (N'_Z, M'_Z).
    $x.Z \leftarrow x.Z * (x.P / x.P^{old}) * x.l$
    $x.P^{old} \leftarrow x.P$
    $x.n_Z \leftarrow N'_Z$
    $x.m_Z \leftarrow M'_Z$
▷ Run the second half of algorithm 2, Advance Trie Time, on B to (N'_Z, M'_Z).
for each node $x < F$ in reverse topological order **do**
    $x.Z \leftarrow \sum_{y \in x.\text{children}} y.\text{Z}$
    $x.n_Z \leftarrow N'_Z$
    $x.m_Z \leftarrow M'_Z$
$B.M_Z \leftarrow M'_Z$
$B.M_P \leftarrow M'_P$
$B.M_{\mathbf{P}} \leftarrow M'_{\mathbf{P}}$

Algorithm 12 Complete Likelihood Update

Require: Bayesian Trie B.

$N'_t \leftarrow B.N_t + 1$
$N'_Z \leftarrow B.N_Z + 1$
$N'_R \leftarrow B.N_R + 1$
$M'_Z \leftarrow B.M_Z$ ▷ In the present algorithms, N'_t, N'_Z, and N'_R remain synchronized.
for each node $x < C^L_{N'_t}$ in topological order **do**
▷ Run algorithm 3, Push Likelihood, on x.
    $y.l \leftarrow y.l * x.l \quad \forall y \in x.\text{children}$
    $x.l \leftarrow 1$
    $x.n_t \leftarrow B.N_t$
for each node $x \in C^L_{N'_t}$ **do**
▷ Run algorithm 4, Apply Likelihood Delta, on frontier node x.
    $x.l \leftarrow x.l * \Delta^L_{N'_t}(x)$
▷ Run the first half of algorithm 2, Advance Trie Time, on B to (N'_Z, M'_Z).
▷ Run algorithm 1, Advance Node Time, on x to (N'_Z, M'_Z).
    $x.Z \leftarrow x.Z * (x.P / x.P^{old}) * x.l$
    $x.P^{old} \leftarrow x.P$
    $x.n_Z \leftarrow N'_Z$
    $x.m_Z \leftarrow M'_Z$
▷ Run algorithm 8, MTCDL Update, on x to time N'_R.
    $\widehat{R}(\mathbf{d}, x) * = x.l \quad$ for all canonical $\mathbf{d} \leq x$
    $x.n_R \leftarrow N'_R$
$B.N_t \leftarrow N'_t$
▷ Run algorithm 7, MTCDL Up Propagation, on B for frontier $C^L_{N'_t}$.
for each node $f \in C^L_{N'_t}$ **do**
    for $i \leq |f|$ **do**
        $\mathbf{b} \leftarrow T(f[:i])$
        for j from $|f|$ down to 0 **do**
            $\widehat{R}(\lfloor f[:j] \rfloor_{\mathbf{b}}, f[:j]) \leftarrow_{\max} \widehat{R}(\mathbf{b}, f)$
for each node $x < C^L_{N'_t}$ **do**
    $x.n_R \leftarrow N'_R$
▷ Run the second half of algorithm 2, Advance Trie Time, on B to (N'_Z, M'_Z).
for each node $x < C^L_{N'_t}$ in reverse topological order **do**
    $x.Z \leftarrow \sum_{y \in x.\text{children}} y.\text{Z}$
    $x.n_Z \leftarrow N'_Z$
    $x.m_Z \leftarrow M'_Z$
$B.N_Z \leftarrow N'_Z$
$B.N_R \leftarrow N'_R$

Algorithm 13 Complete Initialization

Require: New Node Value x.

▷ Initialize the prior

$x.\boldsymbol{P} \leftarrow |\boldsymbol{\Lambda}|^{-|T(x)|}$
$x.m_{\mathbf{P}} \leftarrow 0$
$x.P \leftarrow 0$
for $i \leq |x|$ **do**
    $\mathbf{a} \leftarrow T(x[:i])$
    $x.P \leftarrow x.P + |\boldsymbol{\Lambda}|^{-|\mathbf{a}|} * \Phi_0(\mathbf{a}, x)$
$x.m_P \leftarrow 0$

▷ Initialize the likelihood

$x.l \leftarrow 1$
$x.n_t \leftarrow 0$

▷ Initialize the MTCDL

for $i \leq |x|$ **do**
    $\mathbf{a} \leftarrow T(x[:i])$
    if $(i = 0 \wedge x[:i] \in V_{\leq})$ or $\xi(\mathbf{a}[-1], x[:i])$ **then**
        $\widehat{R}(\mathbf{a}, x) \leftarrow 1$
    else
        $\widehat{R}(\mathbf{a}, x) \leftarrow 0$
$x.n_R \leftarrow 0$

▷ Initialize the posterior

$x.Z \leftarrow x.P$
$x.P^{old} \leftarrow x.P$
$x.n_Z \leftarrow 0$
$x.m_Z \leftarrow 0$

Algorithm 14 Complete Push

Require: Bayesian Trie B.
Require: Node x.
Require: All ancestors of x are correct and have no unpushed likelihood. ▷ Run algorithm 5, Compute Token Branch Prior, on x with new time $B.M_{\mathbf{P}}$.

if $x = \epsilon$ **then**
    $x.\mathbf{P} \leftarrow 1$
else
    $x.\mathbf{P} \leftarrow \pi(x).\mathbf{P} * \mathbf{Q}_{B.M_{\mathbf{P}}}(T(x)[-1] \mid T(x)[:-1])$
$x.m_{\mathbf{P}} \leftarrow B.M_{\mathbf{P}}$ ▷ Run algorithm 6, Compute Branch Prior, on x with new time $B.M_P$.
$x.P \leftarrow \sum_{i=0}^{|x|-1}(x[:i]).\mathbf{P} * \Phi_{B.M_P}(\kappa_x(i), x)$
$x.m_P \leftarrow B.M_P$ ▷ Run algorithm 8, MTCDL Update, on x to time $B.N_R$.
$\widehat{R}(\mathbf{d}, x) * = \prod_{i=x.n_R+1}^{B.N_R} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$ for all canonical $\mathbf{d} \leq x$
$x.n_R \leftarrow B.N_R$ ▷ Run algorithm 3, Push Likelihood, on x.
$y.l \leftarrow y.l * x.l \quad \forall y \in x.\text{children}$
$x.l \leftarrow 1$
$x.n_t \leftarrow B.N_t$

2.7 OLD STUFF

Algorithm 15 Complete Visible Trie

Require: Trie V of visible nodes

```
for each node v ∈ V do
    if v is a leaf in V then
        continue                                   ▷ Skip if it is already a leaf
                                                   ▷ Propagate likelihood to invisible children
    for each character c in the alphabet do
        if (v + c) ∉ V then
            Add (v + c) to V
            Set (v + c).new_likelihood ← v.new_likelihood
return V
```

Prior update:

After receiving an "LLM Result" we must update the trie to reflect this refinement of the language model.

Step 1: Select initial frontier: The "LLM Result" corresponds to a token sequence, $\mathbf{A}$. We choose the corresponding character sequence, S, in the bayesian trie as our initial frontier.

Step 2: Down Propagation: We descend from the initial frontier via valid descent until we reach a valid update leafset. A node is known to belong to the valid update leafset, and no further descent from that node is necessary, if and only if the node satisfies one of the following two cases:

- **Valid Likelihood**: The node has never been a parent to a visible node, i.e., it has a valid likelihood.
- **Space Character**: The node is a space character. Formally, the condition is that there exists some token following upon $\mathbf{A}$ which every possible suffix of our node S, must contain, i.e., we require that S satisfy,

  $$\exists \mathbf{B}, \mathbf{B} > \mathbf{A}, |\mathbf{B}| = |\mathbf{A}| + 1, \forall Y \geq S, \mathbf{B} < Tokenize(Y)$$

  but in practice we implement this by recognizing that this condition is always true of the space character and rarely true otherwise.

Note Bene: Why is descending to the space character necessary? After all, if we know exactly how a node's prior has changed, and we know that its likelihood has not changed, then we should be able to correctly recalculate the unnormalized posterior, just as how in the likelihood update we could recalculate the unnormalized posterior of a node if we knew how its likelihood had changed and that its prior had remained constant. The subtle fallacy in this thinking is that the *likelihood of the node has changed*. Although we often think of the likelihood as being independent of the prior, this is only true for pure hypotheses. For a composite hypothesis, like this node we have,

$$P(D|S) = \sum_i P(D|S_i)P(S_i|S)$$

where S_i are the children of S. Since the weighting, i.e., the prior, has changed, so has the *likelihood* of this node. Thus we do not satisfy case 4 above, because the likelihood has changed.

	Likelihood Update	**Prior Update**
Mathematical Term	$P(D \mid X)$	$P(X)$
Recompute Posteriors	Yes	Yes
Resize Boxes	Yes	Yes
How Many New Nodes?	Dozens	Zero to a few
How to Set Phases	Optimized / gradient descent	Random (or ancestor / minimum placement)
Human-Initiated	Yes	No
Show Immediately	Yes	No — wait for ancestor to finish
Branch of Tree Affected	Entire tree	Single branch

Table 2.2: Comparison of likelihood updates and prior updates.

Chapter 3

Byte-Pair Encoding

This chapter collects the material in this book that depends specifically on byte-pair encoding (BPE). Section 3.1 contains the formal mechanics of stagewise BPE tokenization and the proof of canonical concatenation. Section 3.2 then gives an implementation-oriented view based on merge IDs, left and right spines, and a fast boundary test for token-pair canonicality.

3.1 Byte-Pair Encoding Mechanics

We now specialize the general tokenization formalism of this book to byte-pair encoders. Our goal is to prove a structural property that is stronger than boundary sufficiency and is specific to BPE tokenizers.

Throughout this section we identify each BPE token with its spelling. Thus a token is itself a nonempty string over the character alphabet Λ, and the literalization map is just concatenation of tokens.

Definition 3.1 (Character Tokenization)**.** *For a string $x = x_1 \cdots x_n$, let*

$$\mathrm{chars}(x) = [x_1, \ldots, x_n]$$

denote the sequence of one-character strings whose concatenation is x.

Definition 3.2 (BPE Merge System)**.** *A BPE merge system of depth N consists of vocabularies*

$$\boldsymbol{\Lambda}^{(0)} \subseteq \boldsymbol{\Lambda}^{(1)} \subseteq \cdots \subseteq \boldsymbol{\Lambda}^{(N)}$$

and tokens $\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}, \boldsymbol{\gamma_k}$ for each $1 \le k \le N$ satisfying:

1. $\boldsymbol{\Lambda}^{(0)} = \Lambda$*, where each character is regarded as a one-character token.*
2. $\boldsymbol{\alpha_k}, \boldsymbol{\beta_k} \in \boldsymbol{\Lambda}^{(k-1)}$.
3. $\boldsymbol{\gamma_k} = \boldsymbol{\alpha_k} + \boldsymbol{\beta_k}$ *as strings.*
4. $\boldsymbol{\gamma_k} \notin \boldsymbol{\Lambda}^{(k-1)}$.
5. $\boldsymbol{\Lambda}^{(k)} = \boldsymbol{\Lambda}^{(k-1)} \cup \{\boldsymbol{\gamma_k}\}$.

Definition 3.3 (Single BPE Merge Pass)**.** *For $1 \le k \le N$, the kth merge pass*

$$M_k : (\boldsymbol{\Lambda}^{(k-1)})^* \to (\boldsymbol{\Lambda}^{(k)})^*$$

is defined recursively by

$$M_k([]) = [],$$
$$M_k([\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}] + \mathbf{r}) = [\boldsymbol{\gamma_k}] + M_k(\mathbf{r}),$$

and

$$M_k([\boldsymbol{\tau}] + \mathbf{r}) = [\boldsymbol{\tau}] + M_k(\mathbf{r})$$

whenever either $\mathbf{r} = []$, or the first token of $\mathbf{r}$ is not $\boldsymbol{\beta_k}$, or $\boldsymbol{\tau} \neq \boldsymbol{\alpha_k}$.

Thus M_k performs the left-to-right maximal non-overlapping replacement of $[\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}]$ by $[\boldsymbol{\gamma_k}]$.

Definition 3.4 (Stagewise BPE Tokenizers)**.** *The tokenizer after k merge stages is the map*

$$T_k : S \to (\mathbf{\Lambda}^{(k)})^*$$

defined recursively by

$$T_0(x) = \text{chars}(x), \qquad T_k(x) = M_k(T_{k-1}(x)).$$

The final BPE tokenizer is $T := T_N$.

Definition 3.5 (Stagewise Canonicality)**.** *A token sequence $\mathbf{a} \in (\mathbf{\Lambda}^{(k)})^*$ is stage-k canonical if and only if*

$$T_k(T^{-1}(\mathbf{a})) = \mathbf{a}.$$

We denote the set of stage-k canonical token sequences by $\mathbf{C}^{(k)}$.

Proposition 3.1 (Stage-0 Canonicality)**.** *A token sequence is stage-0 canonical if and only if each of its tokens has length 1.*

Proof. By definition,

$$T_0(T^{-1}(\mathbf{a})) = \text{chars}(T^{-1}(\mathbf{a})).$$

The token sequence on the right is obtained by splitting the literalization of $\mathbf{a}$ into one-character strings. Therefore it is equal to $\mathbf{a}$ if and only if every token of $\mathbf{a}$ is already a one-character string. □

Definition 3.6 (Single-Stage Decompression)**.** *For $1 \le k \le N$, the single-stage decompression map*

$$D_k : (\mathbf{\Lambda}^{(k)})^* \to (\mathbf{\Lambda}^{(k-1)})^*$$

is defined recursively by

$$D_k([]) = [],$$
$$D_k([\boldsymbol{\gamma_k}] + \mathbf{r}) = [\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}] + D_k(\mathbf{r}),$$

and

$$D_k([\boldsymbol{\tau}] + \mathbf{r}) = [\boldsymbol{\tau}] + D_k(\mathbf{r}) \qquad \text{for } \boldsymbol{\tau} \neq \boldsymbol{\gamma_k}.$$

Definition 3.7 (Stage-k Reduced Sequence)**.** *For $1 \le k \le N$, we say that a token sequence $\mathbf{a}$ is stage-k reduced if and only if it contains no adjacent occurrence of the pair $[\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}]$. Equivalently,*

$$\text{red}_k(\mathbf{a}) \iff \nexists i < |\mathbf{a}| - 1,\ \mathbf{a}[i] = \boldsymbol{\alpha_k} \wedge \mathbf{a}[i+1] = \boldsymbol{\beta_k}.$$

Lemma 3.1 (Single-stage decompression preserves literalization)**.** *For every $\mathbf{a} \in (\mathbf{\Lambda}^{(k)})^*$,*

$$T^{-1}(D_k(\mathbf{a})) = T^{-1}(\mathbf{a}).$$

Proof. We argue by induction on $|\mathbf{a}|$.

If $\mathbf{a} = []$, the claim is immediate. Otherwise write $\mathbf{a} = [\boldsymbol{\tau}] + \mathbf{r}$.

If $\boldsymbol{\tau} = \boldsymbol{\gamma_k}$, then

$$D_k(\mathbf{a}) = [\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}] + D_k(\mathbf{r}),$$

so

$$\begin{aligned} T^{-1}(D_k(\mathbf{a})) &= \boldsymbol{\alpha_k} + \boldsymbol{\beta_k} + T^{-1}(D_k(\mathbf{r})) \\ &= \boldsymbol{\gamma_k} + T^{-1}(\mathbf{r}) \\ &= T^{-1}(\mathbf{a}), \end{aligned}$$

using the defining relation $\boldsymbol{\gamma_k} = \boldsymbol{\alpha_k} + \boldsymbol{\beta_k}$ and the inductive hypothesis.

If $\boldsymbol{\tau} \neq \boldsymbol{\gamma_k}$, then

$$D_k(\mathbf{a}) = [\boldsymbol{\tau}] + D_k(\mathbf{r}),$$

and therefore

$$T^{-1}(D_k(\mathbf{a})) = \boldsymbol{\tau} + T^{-1}(D_k(\mathbf{r})) = \boldsymbol{\tau} + T^{-1}(\mathbf{r}) = T^{-1}(\mathbf{a}),$$

again by the inductive hypothesis. □

Lemma 3.2 (Single-stage decompression respects concatenation)**.** *For all token sequences* $\mathbf{a}$ *and* $\mathbf{b}$*,*

$$D_k(\mathbf{a}+\mathbf{b}) = D_k(\mathbf{a}) + D_k(\mathbf{b}).$$

Proof. We argue by induction on $|\mathbf{a}|$.

If $\mathbf{a} = []$, the claim is immediate. Otherwise write $\mathbf{a} = [\boldsymbol{\tau}] + \mathbf{r}$ and split on whether $\boldsymbol{\tau} = \boldsymbol{\gamma_k}$. In each case the recursive definition of D_k reduces the claim to the inductive hypothesis. □

Lemma 3.3 (Decompression inverts a merge pass)**.** *For every* $\mathbf{a} \in (\boldsymbol{\Lambda}^{(k-1)})^*$*,*

$$D_k(M_k(\mathbf{a})) = \mathbf{a}.$$

Proof. We argue by induction on $|\mathbf{a}|$.

If $\mathbf{a} = []$, the claim is immediate. Otherwise write $\mathbf{a} = [\boldsymbol{\tau}] + \mathbf{r}$.

If $\boldsymbol{\tau} = \boldsymbol{\alpha_k}$ and $\mathbf{r} = [\boldsymbol{\beta_k}] + \mathbf{r}'$, then

$$M_k(\mathbf{a}) = [\boldsymbol{\gamma_k}] + M_k(\mathbf{r}'),$$

so

$$D_k(M_k(\mathbf{a})) = [\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}] + D_k(M_k(\mathbf{r}')) = [\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}] + \mathbf{r}' = \mathbf{a},$$

by the inductive hypothesis.

Otherwise,

$$M_k(\mathbf{a}) = [\boldsymbol{\tau}] + M_k(\mathbf{r}),$$

and since $\boldsymbol{\tau} \neq \boldsymbol{\gamma_k}$,

$$D_k(M_k(\mathbf{a})) = [\boldsymbol{\tau}] + D_k(M_k(\mathbf{r})) = [\boldsymbol{\tau}] + \mathbf{r} = \mathbf{a},$$

again by the inductive hypothesis. □

Lemma 3.4 (Merge pass yields a reduced sequence)**.** *For every* $\mathbf{a} \in (\boldsymbol{\Lambda}^{(k-1)})^*$*,*

$$\mathrm{red}_k(M_k(\mathbf{a})).$$

Proof. We argue by induction on $|\mathbf{a}|$.

If $\mathbf{a} = []$, the claim is immediate. Otherwise write $\mathbf{a} = [\boldsymbol{\tau}] + \mathbf{r}$.

If $\boldsymbol{\tau} = \boldsymbol{\alpha_k}$ and $\mathbf{r} = [\boldsymbol{\beta_k}] + \mathbf{r}'$, then

$$M_k(\mathbf{a}) = [\boldsymbol{\gamma_k}] + M_k(\mathbf{r}').$$

By the inductive hypothesis, the suffix $M_k(\mathbf{r}')$ is stage-k reduced. Since the first token is $\boldsymbol{\gamma_k}$ rather than $\boldsymbol{\alpha_k}$, the whole sequence is stage-k reduced.

Otherwise,

$$M_k(\mathbf{a}) = [\boldsymbol{\tau}] + M_k(\mathbf{r}).$$

By the inductive hypothesis, the suffix $M_k(\mathbf{r})$ is stage-k reduced. It remains only to rule out an adjacent occurrence of $[\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}]$ starting at the first token.

If $\boldsymbol{\tau} \neq \boldsymbol{\alpha_k}$, this is immediate. If $\boldsymbol{\tau} = \boldsymbol{\alpha_k}$, then by hypothesis the first token of $\mathbf{r}$ is either absent or different from $\boldsymbol{\beta_k}$. Hence the first token of $M_k(\mathbf{r})$ is also either absent or different from $\boldsymbol{\beta_k}$. Therefore no adjacent occurrence of $[\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}]$ starts at the first token. □

Lemma 3.5 (Reduced sequences are reconstructed by decompression)**.** *If* $\mathrm{red}_k(\mathbf{a})$*, then*

$$M_k(D_k(\mathbf{a})) = \mathbf{a}.$$

Proof. We argue by induction on $|\mathbf{a}|$.

If $\mathbf{a} = []$, the claim is immediate. Otherwise write $\mathbf{a} = [\boldsymbol{\tau}] + \mathbf{r}$.

If $\boldsymbol{\tau} = \boldsymbol{\gamma_k}$, then

$$D_k(\mathbf{a}) = [\boldsymbol{\alpha_k}, \boldsymbol{\beta_k}] + D_k(\mathbf{r}),$$

and hence

$$M_k(D_k(\mathbf{a})) = [\boldsymbol{\gamma_k}] + M_k(D_k(\mathbf{r})) = [\boldsymbol{\gamma_k}] + \mathbf{r} = \mathbf{a},$$

by the inductive hypothesis.

If $\boldsymbol{\tau} \neq \boldsymbol{\gamma_k}$, then

$$D_k(\mathbf{a}) = [\boldsymbol{\tau}] + D_k(\mathbf{r}).$$

Because $\mathrm{red}_k(\mathbf{a})$, either $\boldsymbol{\tau} \neq \boldsymbol{\alpha_k}$, or $\mathbf{r} = []$, or the first token of $\mathbf{r}$ is not $\boldsymbol{\beta_k}$. In all cases the first clause in the definition of M_k does not apply to $[\boldsymbol{\tau}] + D_k(\mathbf{r})$, so

$$M_k(D_k(\mathbf{a})) = [\boldsymbol{\tau}] + M_k(D_k(\mathbf{r})) = [\boldsymbol{\tau}] + \mathbf{r} = \mathbf{a},$$

again by the inductive hypothesis. □

Theorem 3.1 (Stagewise canonicality characterization)**.** *Let $1 \leq k \leq N$. Then for every token sequence $\mathbf{a} \in (\boldsymbol{\Lambda}^{(k)})^*$,*

$$\mathbf{a} \in \mathbf{C}^{(k)} \iff \mathrm{red}_k(\mathbf{a}) \wedge D_k(\mathbf{a}) \in \mathbf{C}^{(k-1)}.$$

Proof. We prove both implications.

First suppose $\mathbf{a} \in \mathbf{C}^{(k)}$. Then

$$\mathbf{a} = T_k(T^{-1}(\mathbf{a})) = M_k(T_{k-1}(T^{-1}(\mathbf{a}))).$$

Therefore $\mathrm{red}_k(\mathbf{a})$ by lemma 3.4.

Also,

$$D_k(\mathbf{a}) = D_k(M_k(T_{k-1}(T^{-1}(\mathbf{a})))) = T_{k-1}(T^{-1}(\mathbf{a}))$$

by lemma 3.3. By lemma 3.1,

$$T^{-1}(D_k(\mathbf{a})) = T^{-1}(\mathbf{a}),$$

so

$$T_{k-1}(T^{-1}(D_k(\mathbf{a}))) = D_k(\mathbf{a}).$$

Hence $D_k(\mathbf{a}) \in \mathbf{C}^{(k-1)}$.

Conversely, suppose

$$\mathrm{red}_k(\mathbf{a}) \qquad \text{and} \qquad D_k(\mathbf{a}) \in \mathbf{C}^{(k-1)}.$$

Then

$$T_{k-1}(T^{-1}(D_k(\mathbf{a}))) = D_k(\mathbf{a}).$$

By lemma 3.1,

$$T^{-1}(D_k(\mathbf{a})) = T^{-1}(\mathbf{a}),$$

so

$$T_k(T^{-1}(\mathbf{a})) = M_k(T_{k-1}(T^{-1}(\mathbf{a}))) = M_k(D_k(\mathbf{a})).$$

Since $\mathrm{red}_k(\mathbf{a})$, lemma 3.5 gives

$$M_k(D_k(\mathbf{a})) = \mathbf{a}.$$

Therefore

$$T_k(T^{-1}(\mathbf{a})) = \mathbf{a},$$

i.e. $\mathbf{a} \in \mathbf{C}^{(k)}$. □

Theorem 3.2 (BPE overlap concatenation theorem)**.** *For every $k \in \{0, \ldots, N\}$, if*

$$\mathbf{b} \neq \boldsymbol{\epsilon}, \qquad \mathbf{a} + \mathbf{b} \in \mathbf{C}^{(k)} \qquad \textit{and} \qquad \mathbf{b} + \mathbf{c} \in \mathbf{C}^{(k)},$$

then

$$\mathbf{a} + \mathbf{b} + \mathbf{c} \in \mathbf{C}^{(k)}.$$

Proof. We argue by induction on k.

For $k = 0$, proposition 3.1 implies that every token occurring in $\mathbf{a}+\mathbf{b}$ and every token occurring in $\mathbf{b}+\mathbf{c}$ has length 1. Hence every token in $\mathbf{a}+\mathbf{b}+\mathbf{c}$ has length 1, so $\mathbf{a}+\mathbf{b}+\mathbf{c} \in \mathbf{C}^{(0)}$ by the same proposition.

Now let $1 \le k \le N$, and suppose the theorem has been proved for $k-1$. Assume

$$\mathbf{b} \neq \boldsymbol{\epsilon}, \qquad \mathbf{a}+\mathbf{b} \in \mathbf{C}^{(k)} \qquad \text{and} \qquad \mathbf{b}+\mathbf{c} \in \mathbf{C}^{(k)}.$$

By theorem 3.1,

$$\mathrm{red}_k(\mathbf{a}+\mathbf{b}), \quad D_k(\mathbf{a}+\mathbf{b}) \in \mathbf{C}^{(k-1)}, \quad \mathrm{red}_k(\mathbf{b}+\mathbf{c}), \quad D_k(\mathbf{b}+\mathbf{c}) \in \mathbf{C}^{(k-1)}.$$

By lemma 3.2,

$$D_k(\mathbf{a}+\mathbf{b}) = D_k(\mathbf{a}) + D_k(\mathbf{b}), \qquad D_k(\mathbf{b}+\mathbf{c}) = D_k(\mathbf{b}) + D_k(\mathbf{c}).$$

Applying the inductive hypothesis at stage $k-1$ yields

$$D_k(\mathbf{a}) + D_k(\mathbf{b}) + D_k(\mathbf{c}) \in \mathbf{C}^{(k-1)}.$$

Using lemma 3.2 once more,

$$D_k(\mathbf{a}+\mathbf{b}+\mathbf{c}) \in \mathbf{C}^{(k-1)}.$$

It remains to prove that $\mathbf{a}+\mathbf{b}+\mathbf{c}$ is stage-k reduced. Because $\mathbf{b} \neq \boldsymbol{\epsilon}$, the boundary between $\mathbf{a}$ and $\mathbf{c}$ does not create an adjacent pair in $\mathbf{a}+\mathbf{b}+\mathbf{c}$. Every adjacent pair of tokens in $\mathbf{a}+\mathbf{b}+\mathbf{c}$ lies either in the prefix $\mathbf{a}+\mathbf{b}$ or in the suffix $\mathbf{b}+\mathbf{c}$. Since both of those sequences are stage-k reduced, so is $\mathbf{a}+\mathbf{b}+\mathbf{c}$.

A final application of theorem 3.1 gives

$$\mathbf{a}+\mathbf{b}+\mathbf{c} \in \mathbf{C}^{(k)},$$

as required. □

Corollary 3.1 (BPE last-pair extension)**.** *Assume the tokenization function of this chapter is induced by a BPE merge system, so that* $\mathbf{C} = \mathbf{C}^{(N)}$*. Let* $\mathbf{a} \in \mathbf{C}$ *be nonempty and let* $\boldsymbol{\tau}$ *be a token. If*

$$\psi(\mathbf{a}[-1], \boldsymbol{\tau}),$$

then

$$\mathbf{a}+\boldsymbol{\tau} \in \mathbf{C}.$$

Proof. Let

$$\mathbf{a}' := \mathbf{a}[:-1], \qquad \mathbf{b} := [\mathbf{a}[-1]], \qquad \mathbf{c} := [\boldsymbol{\tau}].$$

Then

$$\mathbf{a}'+\mathbf{b} = \mathbf{a} \in \mathbf{C}.$$

Also, in the BPE specialization, the condition

$$\psi(\mathbf{a}[-1], \boldsymbol{\tau})$$

is exactly the statement that

$$\mathbf{b}+\mathbf{c} \in \mathbf{C}.$$

Applying theorem 3.2 with $k = N$ gives

$$\mathbf{a}'+\mathbf{b}+\mathbf{c} \in \mathbf{C},$$

i.e.

$$\mathbf{a}+\boldsymbol{\tau} \in \mathbf{C}.$$

□

3.2 Spines and Pair Canonicality

For implementation purposes it is useful to represent a BPE merge rule as a tuple

$$(\iota_1, \iota_2) \mapsto (\iota_3, r),$$

where ι_1 and ι_2 are the IDs of the two child tokens, ι_3 is the ID of their merged token, and r is the merge rank. Lower rank means higher priority. In code, the merge table is keyed by the child pair (ι_1, ι_2) and returns the merged ID ι_3 together with its rank r.

Definition 3.8 (Left and right spines)**.** *Let $\boldsymbol{\tau}$ be a BPE token, and let its binary merge tree be the unique tree whose leaves are characters and whose internal nodes are merge results, with root $\boldsymbol{\tau}$. The* left spine *of $\boldsymbol{\tau}$ is the leftmost root-to-leaf path, reported from leaf to root. The* right spine *is defined symmetrically.*

In the implementation we store either spine as a list of pairs

$$[(\iota_0, r_0), (\iota_1, r_1), \ldots, (\iota_m, r_m)],$$

where:

1. ι_0 is the ID of the boundary-adjacent character on that side;
2. ι_m is the ID of the whole token;
3. for $j < m$, the rank r_j is the merge rank of the operation that grows ι_j into ι_{j+1} along that spine;
4. $r_m = \texttt{None}$.

Thus, for a token whose derivation is

$$h + e \to he, \qquad l + l \to ll, \qquad he + ll \to hell,$$

the left spine is

$$[(h, \mathrm{rank}(h, e)), (he, \mathrm{rank}(he, ll)), (hell, \texttt{None})],$$

while the right spine is

$$[(l, \mathrm{rank}(l, l)), (ll, \mathrm{rank}(he, ll)), (hell, \texttt{None})].$$

Definition 3.9 (Spine frontier)**.** *Let $\boldsymbol{\tau_1}$ and $\boldsymbol{\tau_2}$ be tokens. Suppose we are given the right spine of $\boldsymbol{\tau_1}$ and the left spine of $\boldsymbol{\tau_2}$, both stored leaf-to-root as above. A* spine frontier state *is a pair of indices (i, j) pointing into these two lists. The currently exposed boundary tokens are the IDs carried by those two entries.*

Proposition 3.2 (Spine-based canonicality algorithm)**.** *Let $\boldsymbol{\tau_1}$ and $\boldsymbol{\tau_2}$ be BPE tokens. Assume their right and left spines are known. Consider the following procedure.*

Initialize two pointers at the leaf ends of the spines. At each step examine three candidate ranks:

1. *the rank of the next merge on the right spine of $\boldsymbol{\tau_1}$;*
2. *the rank of the next merge on the left spine of $\boldsymbol{\tau_2}$;*
3. *the rank of the merge of the two currently exposed boundary tokens, if such a merge exists in the BPE merge table.*

If the third rank is the unique highest-priority rank, the boundary is crossed and the pair is not canonical. If the first or second rank has highest priority, advance the corresponding spine pointer to the next larger spine token. If all three ranks are absent, terminate and return that the pair is canonical.

This procedure returns `true` *if and only if*

$$\psi(\boldsymbol{\tau_1}, \boldsymbol{\tau_2}).$$

Proof. The key invariant is that after all merges of rank smaller than the current frontier have been applied, the exposed token on the left side of the boundary is exactly the current node of the right spine of $\boldsymbol{\tau_1}$, and the exposed token on the right side of the boundary is exactly the current node of the left spine of $\boldsymbol{\tau_2}$.

Any merge that is strictly internal to the left token and affects the boundary must enlarge the boundary-adjacent suffix of $\boldsymbol{\tau_1}$, and therefore must be the next merge on its right spine. Likewise, any boundary-relevant

internal merge on the right token must be the next merge on its left spine. The only other possible boundary-relevant event is a direct merge of the two currently exposed boundary tokens. Therefore every event that can affect whether the boundary survives is represented by exactly one of the three candidate ranks listed above.

If the cross-boundary rank is the highest-priority one, then the next relevant event merges the two boundary tokens together, so the boundary does not survive and the pair is not canonical. If instead one of the within-token spine ranks is the highest-priority one, then the corresponding token grows inward-to-outward along its spine while the boundary is preserved, so advancing that pointer is correct. Finally, if all three ranks are absent, no further merge can affect the boundary, so the boundary survives to the end of tokenization and the pair is canonical. □

The resulting decision procedure is linear in the two spine lengths:

$$O(|\,\mathrm{rspine}(\boldsymbol{\tau_1})| + |\,\mathrm{lspine}(\boldsymbol{\tau_2})|),$$

with one merge-table lookup per iteration. In practice this is substantially cheaper than retokenizing the concatenated string $\rho(\boldsymbol{\tau_1}) + \rho(\boldsymbol{\tau_2})$ from scratch.

3.2.1 Implementation Optimizations

The proposition above only describes the logical decision procedure. In a high-throughput setting, especially when one fixed first token must be tested against many possible second tokens, the main engineering work lies in lowering the constant factors of this frontier walk. The optimizations that proved most effective in benchmarking were the following:

1. **Prepared-first dense tables.** For one fixed first token, precompute the dense cross-rank arrays needed by its right-spine nodes and reuse them across all candidate second tokens, rather than performing a fresh merge-table lookup on every frontier step.
2. **Raw `u16` `rank_plus_one` state.** Store ranks in a sentinel encoding with `0` meaning "no rank" and positive entries meaning `rank + 1`. This avoids repeated `Option`-level branching in the hot loop.
3. **Unchecked indexing in the hot path.** Once the spine-length and ID bounds are established by construction, the inner loop can use unchecked array access. This removes a measurable amount of bounds-check overhead from the per-candidate kernel.
4. **Compact left-spine representation.** Represent each candidate second token by fixed-size arrays of boundary IDs and `rank_plus_one` values, rather than by a more general vector-like structure. This keeps the frontier state small and cache-friendly.
5. **Per-first-token row pointers and right-rank arrays.** During preparation, extract the right-spine ranks and pointers to the relevant dense cross-rank rows for the current first token. The kernel can then load exactly the few scalar values it needs at each frontier step.
6. **Specialization by exact left-spine length.** Bucket the candidate second tokens by exact left-spine length and dispatch to length-specialized kernels. Since the spines are very short, this removes a substantial amount of generic control-flow overhead.
7. **A shared zero row for empty prepared rows.** When a prepared right-spine node has no possible merge partners, point it at one global zero-filled dense row. This preserves a branch-free inner loop while avoiding needless per-token allocation for empty rows.
8. **Lexicographic ordering by left-spine ID sequence.** Process candidate second tokens in lexicographic order of their left-spine ID tuples. This improves locality for the earliest frontier states, which are visited most often, and reduces effective cache pressure on the dense cross-rank rows.
9. **Swapped-tight dense layout for unknown frontier index.** At each step the kernel needs values of the form

   `cross_rank[left_id, i]`

   where $left\_id$ is known from the second-token spine, but the needed right-spine index i is not known in advance because it depends on prior rank comparisons. A layout that keeps all i values for one fixed

left ID close together is therefore preferable. Concretely, store cross-ranks in left-ID-major order with stride equal to the *actual* right-spine length of the prepared first token ("swapped-tight").

10. **Lookahead prefetch on swapped-tight rows.** Because the dominant stall risk is memory latency on future cross-rank row touches, issue a short lookahead prefetch over upcoming left-spine IDs. In swapped-tight layout this prefetch targets compact left-ID rows, so one prefetch often covers several candidate i loads for that row. In practice this is treated as a policy knob (by bucket and workload), not as a correctness requirement.

These optimizations do not change the correctness argument of Proposition 3.2; they only refine the data layout and the order in which the frontier states are evaluated. The resulting implementation still performs the same logical three-way comparison at each frontier state, but with much lower constant cost.

Chapter 4

Determine Visible Nodes

In this step we identify which prefixes have sufficient posterior probability to be visually displayed.

The two questions are which prefixes to show and whether to show them immediately or to wait until showing them.

4.1 Setting the Visibility Threshold

The visibility threshold will determine the number of visible nodes, which entails a trade-off.

Advantages of a high threshold and few visible nodes:

- There is less clutter and it is less overwhelming to the user.
- Nodes are larger, avoiding the need for small fonts and timers. The user can recognize timers more easily.
- Faster computation and rendering.

Advantages of a low threshold and many visible nodes:

- More granular targets allows us to spread out the probability more evenly, i.e., we can set the timers to make a better approximation to the uniform distribution. (Refer to chapter 6.)

Thus, we want to set the visibility threshold as high as we can while still being able set the timers to approximate a uniform distribution.

4.2 Searching for Visible Nodes

In order to identify all nodes exceeding the visibility threshold, we descend the trie. We know that posterior probability decreases monotonically with depth, hence we may stop descending upon reaching a node whose (normalized) posterior probability is less than the visibility threshold. It is necessary to descend the trie in the manner described in chapter 2, since this guarantees that the unnormalized posterior probabilities we observe are valid. (Searching for visible nodes with a flat scan over all nodes would be invalid, since many of their posteriors would be stale, since their updates have been deferred.)

4.3 Responding to Likelihood Updates versus Prior Updates

Every time our trie is updated, we need to recalculate the visible nodes. Thus the set of visible nodes will change in response to prior updates as well as in response to likelihood updates. However, these should not be handled in the same way. Since a likelihood update is triggered by the user's gesture, the user expects that the visible nodes will change in response. On the other hand, a prior update is triggered by the background evaluation of a language model. Immediately redrawing in response to these updates would be jarring and disorienting. It could be especially problematic if a child appeared right before the user was about to select

its parent, causing the user to make an error (select the parent instead of the child), through no fault of their own. Similarly, it would be problematic if a node disappeared right as the user was about to select it.

To mitigate this issue, we enforce a couple of rules about redrawing the trie in response to prior updates:

- A new visible node cannot be drawn while its parent is close to culmination.
- A visible node can not be removed until the next likelihood update.

Chapter 5

Render Trie

5.1 Scrolling

This implementation of Dotter is designed for typing medium-length strings of text of about 400 characters or less. It is intended for typing text messages, emails, chat-bot commands, and search queries. These are typically a few sentences long.

This raises the problem of scrolling, because 400 characters is far too much to fit horizontally on a single screen. Scrolling must be automatic in order not to slow down the user. The advantage of scrolling is that keeps the active part of the trie centered and maximizes utilization of real estate. The disadvantage of scrolling is that it is jarring and disorienting. Below we discuss a simple heuristic to balance this trade-off.

5.1.1 A Quadratic Cost Heuristic

We assume that all characters occupy constant width. We formalize our goal of keeping the active part of the trie centered in terms of the "first fork." The first fork node is the first node that has multiple children in the visible trie. We denote its depth as d. We define the scroll offset s as the number of characters which are off-screen to the left of the window.

Our centering objective means that we want to minimize the distance between the "first fork position", $p := w(d - s)$, where d is the depth of the first fork node and w is the width of any node which is a single parent, and the desired first fork position, p^*. Our stability objective means that we want to minimize changes in the offset between iterations, $\Delta s = s_t - s_{t-1}$.

We may parameterize such an objective with rates a and b, and solve for s_t^*,

$$J = a(\frac{p^* - p}{w})^2 + b\Delta s^2$$

$$J = a(s_t - (d - \frac{p^*}{w}))^2 + b(s_t - s_{t-1})^2$$

$$\implies 0 = 2a(s_t - (d - \frac{p^*}{w})) + 2b(s_t - s_{t-1})$$

$$\implies s_t^* = \frac{a(d - \frac{p^*}{w}) + bs_{t-1}}{a + b}$$

5.1.2 Two Approaches to Backspace

There are two ways to allow the user to go back to nodes that are beyond the scroll window. Either we can have a single "backspace" node that contributes likelihood to all off-screen predecessors, or we can try to actually assign independent timers to every off-screen predecessor. Not only is the former less cluttered, it also has the advantage of showing incremental progress.

5.2 Embellishments

5.2.1 Branches of the Tree

Chapter 6

Stimulus Optimization: Ideal Timer Placement

6.1 The Continuous One Bit Channel

6.1.1 Two Interpretations

The continuous one bit channel can be conceived in two ways: as a continuous channel with a one bit range, or as a series of times of identical events. The equivalency follows from the fact that the continuous channel may be characterized by the times at which its signal changes. To have a finite (differential) capacity we must constrain the channel to a minimum duration of signal, or equivalently, a minimum time between events. For our purpose, we prefer the latter interpretation of the one bit channel, since the user communicates using a sequence of identical events such as blinks or taps.

6.1.2 The Ideal Timing Distribution

What then, is the capacity of the one bit channel? We prefer the event interpretation for analysis. We consider the ideal distribution of the duration between events. Let T be the minimum duration between events and X be the additional delay beyond the minimum, so that the total duration between events is $T + X$. Let $A = T + E[X]$ be the expected inter-arrival time. The rate is given by

$$C = \max_{\mathcal{P}_X} \frac{h(X)}{A}.$$

We refer to the rate-maximizing distribution of X, $\mathcal{P}_X$, as the ideal timing distribution. We will proceed to show that the ideal timing distribution is an exponential distribution.

We treat the capacity, expected value, and differential entropy as functionals of the probability density function $f(x)$. To find the optimal distribution, we introduce a Lagrange multiplier μ for the normalization constraint $\int f(x)dx = 1$ and define the functional $J(f)$:

$$J(f) = \frac{h(X)}{A} - \mu \left(\int f(x)dx - 1 \right)$$

We maximize $J(f)$ by setting the variational derivative with respect to $f(x)$ to zero:

$$\frac{\delta J}{\delta f(x)} = 0$$

Using the quotient rule for variational derivatives, and noting that $\frac{\delta h(X)}{\delta f(x)} = -1 - \ln f(x)$ and $\frac{\delta A}{\delta f(x)} = \frac{\delta E[X]}{\delta f(x)} = x$, we have:

$$\frac{\delta}{\delta f(x)} \left(\frac{h(X)}{A} \right) = \frac{A(-1 - \ln f(x)) - h(X)x}{A^2}$$

Substituting this into the stationarity condition:

$$\frac{A(-1-\ln f(x))-h(X)x}{A^2}-\mu=0$$

Solving for $f(x)$:

$$f(x)\propto\exp\left(-\frac{h(X)}{A}x\right)$$

proving that X is exponentially distributed with rate parameter equal to the rate of the channel (i.e., $\lambda^*=C$). This fact lets us solve for the rate,

$$C=\frac{1-\ln C}{T+1/C}$$

which by algebra has the solution,

$$C=W(T)/T$$

, where W is the Lambert W function.

A Geometric Intuition

The above result (that the ideal timing distribution is exponential) may be appreciated more readily with a geometric argument. We consider a large sample, $\mathbf{X}$, in N-dimensional space, representing the sequence of additional delays X_i. By the law of large numbers, its L_1 norm, $\|\mathbf{X}\|_1=\sum_i X_i$, must be tightly distributed about the N-simplex, $\|\mathbf{X}\|_1=NE[X]$. This can be thought of as the permissible region for $\mathbf{X}$. Thus the entropy maximizing distribution for $\mathbf{X}$ should assign a uniform distribution over the N-simplex, which is precisely what the exponential distribution does as it depends only on the L_1 norm, $f(\mathbf{X})\propto\exp(-\lambda\|\mathbf{X}\|_1)$.

(Incidentally, this conception also provides a geometric proof of the fact that the differential entropy of the exponential distribution is $1-\ln\lambda$, since the entropy of a uniform distribution over the N-simplex should be the logarithm of its volume. Since its side length is N/λ, its volume is $(N/\lambda)^N/N!$, and its entropy per sample (by Stirling's approximation) is

$$h(X)=\lim_{N\to\infty}\frac{1}{N}\left(N(\log(N-\log\lambda)-N(\log N-1))\right)=1-\ln\lambda$$

as expected.)

6.2 Periodic Signals

6.2.1 Random Scan Times

6.2.2 Periodic signals

6.2.3 Approximating A Uniform Distribution with a Gaussian Mixture

UNFINISHED

We have considered the ideal timing distribution and the periodic timing distribution. How can we induce a uniform distribution on the user's signal? There are several targets that the user may aim for, corresponding to all the prefixes displayed. Each target has an assigned signal time, μ_i and a fixed and known (mixing) probability, π_i. Given the noise $\mathcal{E}\sim\mathcal{N}(0,\sigma)$, the distribution of the observed signal is a mixture of gaussians with means mu_i and constant variance σ^2. Given the mixing probabilities, π_i, how should we position the μ_i to maximize the entropy of the signal?

Intuitively we want to space apart the μ_i to approximate a uniform distribution. Those components with higher pi_i should receive more space.

As soon as we receive a signal we recompute the posterior probabilities of the targets and must assign new phases to them. Hence this computation is on the critical path and ought to be computed in a few

milliseconds. This rules out heavier techniques like gradient descent on a numerical approximation of the entropy.

We discuss two heuristic approaches to the problem as well as an approximation for the entropy which may computed efficiently.

Two Heuristics

UNFINISHED We wish to space the components of the mixture as to approximate a uniform distribution. One heuristic is to determine (by binary search) a value C such that

$$\sum_i 2w_i = P$$

where the widths allotted to each component, w_i, are given by

$$w_i = \mathrm{Re}\left(\sigma\sqrt{2\left(\ln\left(\frac{\pi_i}{\sqrt{2\pi}\sigma}\right) - C\right)}\right)$$

where P is the length of the interval. The means are then given by $\mu_i = w_i + \sum_{j=1}^{i-1} 2w_j$.

This heuristic is derived from approximating the entropy of the mixture as $h(X) \approx -\ln f_{\min}$ where $f_{\min}$ is the minimum value of the mixture density, and approximating the mixture by its greatest component at a given point,

$$f(x) \approx \max_i f_i(x)$$

The value of C represents the minimum log density of the approximated mixture, i.e., the minimum over the interval of the maximum log density of the components. The width $2w_i$ is the range over which the ith component may provide a log density greater than C. If $\ln f_i(\mu_i) < C$, where $f_i(\mu_i)$ denotes the maximum density of the ith component at $x = 0$, then clearly $w_i = 0$, otherwise it is as given above. Our algorithm thus maximizes the minimum density of the maximum component over the interval. Although somewhat crude, this heuristic is efficient.

A refinement may be made by instead approximating the mixture by the sum of its two greatest components, and continuing to try to maximize the minimum density of the distribution. In this case, adjacent components should be spaced apart such that the log of the sum of their densities attains a minimum value of C. Given two adjacent components, i and $i+1$, we are interested in their separation, l_i. Letting $\mu_i = 0$ and $\mu_{i+1} = l_i$, we may solve for the minimum value of the two-component mixture:

$$f(x) = \pi_i \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right) + \pi_{i+1} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-l_i)^2}{2\sigma^2}\right)$$

Note that completing the square for the sum of squared distances gives:

$$(x-\mu_1)^2 + (x-\mu_2)^2 = 2\left(x - \frac{\mu_1+\mu_2}{2}\right)^2 + \frac{(\mu_1-\mu_2)^2}{2}$$

Taking the derivative with respect to x and setting it to zero we have,

$$\frac{d}{dx} f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \left(\frac{-2x\pi_i}{2\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right) - \frac{-2(x-l_i)\pi_{i+1}}{2\sigma^2} \exp\left(-\frac{(x-l_i)^2}{2\sigma^2}\right)\right) = 0$$

Algebra yields,

$$\ln\left(\frac{Rx}{l_i - x}\right) = \frac{x l_i}{\sigma^2} - \frac{l_i^2}{2\sigma^2}$$

, where $R = \pi_i/\pi_{i+1}$. Taken with the condition that $\ln f(x) = C$ the two variables, C and R, implicitly determine l_i. So, using a precomputed table of l_i values for a range of C and R values, we can proceed as before, using binary search to find the value of C such that $\sum_i l_i = P$. As before, we will exclude components with $\ln f_i(\mu_i) < C$ from calculations. As long as each adjacent component has a maximum log density greater than C, it will be possible to space them such that the log of their sum attains a minimum value of C.

Approximations to the entropy based on pairwise distances

In order to optimize the placement of the means to maximize entropy, we may minimize a proxy objective J based on pairwise distances between the components. We consider approximations to the entropy for both Gaussian and von Mises mixture models.

Gaussian Mixture Model with Rényi Entropy The Rényi 2-entropy, also known as collision entropy, is defined as $H_2(X) = -\ln \int p(x)^2 dx$. Maximizing this entropy is equivalent to minimizing the information potential $J = \int p(x)^2 dx$. For a mixture of Gaussians $p(x) = \sum_{i=1}^{N} \pi_i \mathcal{N}(x|\mu_i, \sigma^2)$, the integral of the squared density is:

$$J = \int \left(\sum_{i=1}^{N} \pi_i \mathcal{N}(x|\mu_i, \sigma^2) \right)^2 dx = \sum_{i=1}^{N} \sum_{j=1}^{N} \pi_i \pi_j \int \mathcal{N}(x|\mu_i, \sigma^2) \mathcal{N}(x|\mu_j, \sigma^2) dx$$

The integral of the product of two Gaussians is itself a Gaussian evaluated at the difference of their means:

$$\int \mathcal{N}(x|\mu_i, \sigma^2) \mathcal{N}(x|\mu_j, \sigma^2) dx = \mathcal{N}(0|\mu_i - \mu_j, 2\sigma^2) = \frac{1}{\sqrt{4\pi\sigma^2}} \exp\left(-\frac{(\mu_i - \mu_j)^2}{4\sigma^2} \right)$$

By dropping the leading scalar and the self-interaction terms ($i = j$) since they are constant with respect to the optimal placement of the means, the objective to minimize simplifies to the pairwise sum:

$$J = \sum_{i<j} \pi_i \pi_j \exp\left(-\frac{(\mu_i - \mu_j)^2}{4\sigma^2} \right)$$

Gaussian Mixture Model with Kolchinsky Objective Kolchinsky and Tracey [1] introduced a family of computationally efficient estimators for mixture entropy that rely on calculating pairwise distances between component distributions, proving they form a strict lower bound on the true mixture entropy. For homoskedastic Gaussian mixtures, maximizing this lower bound utilizing the Bhattacharyya distance yields a similar pairwise objective. The effective variance in the penalty term is simply doubled compared to the Rényi objective:

$$J = \sum_{i<j} \pi_i \pi_j \exp\left(-\frac{(\mu_i - \mu_j)^2}{8\sigma^2} \right)$$

Von Mises Mixture Model with Rényi Entropy When the components are distributed on the unit circle as von Mises distributions with homoskedastic concentration κ, we again aim to minimize the integral of the squared density. The integral of the cross-term of two von Mises components evaluates to a modified Bessel function of the first kind of order zero, I_0. Stripping out constants and self-terms, the objective becomes:

$$J = \sum_{i<j} \pi_i \pi_j I_0\left(2\kappa \cos\left(\frac{\mu_i - \mu_j}{2} \right) \right)$$

Von Mises Mixture Model with Kolchinsky Objective Applying Kolchinsky's lower bound using the Bhattacharyya coefficient to the von Mises mixture model yields an objective with an identical structure to the Rényi case. The difference lies solely in the concentration parameter, which is halved:

$$J = \sum_{i<j} \pi_i \pi_j I_0\left(\kappa \cos\left(\frac{\mu_i - \mu_j}{2} \right) \right)$$

An approximation for the entropy via Cramér's theorem

UNFINISHED Here we discuss a closed form approximation for the entropy in terms of the mixing weights, π_i, and the chosen means, μ_i. This enables efficient gradient descent on the means μ_i. We take a geometric approach and consider a large sample, $\mathbf{X}$, in N-dimensional space. Let A_i be a discrete random variable

taking on the values of μ_i, with respective probabilities π_i. Let $\mathcal{E}_i \sim \mathcal{N}(0, \sigma^2)$ be random noise on the ith component. Then we have that

$$\mathbf{X} = \mathbf{A} + \mathcal{E}$$

And

$$\begin{aligned} H(\mathbf{X}) &= H(\mathbf{A}, \mathbf{X}) - H(\mathbf{A}|\mathbf{X}) \\ &= H(\mathbf{A}, \mathcal{E}) - H(\mathbf{A}|\mathbf{X}) \quad \text{If } \mathbf{A} \text{ is fixed, } \mathbf{X} \text{ and } \mathcal{E} \text{ differ only be a translation} \\ &= H(\mathcal{E}) + H(\mathbf{A}) - H(\mathbf{A}|\mathbf{X}) \\ &= H(\mathcal{E}) + I(\mathbf{A}; \mathbf{X}) \end{aligned}$$

Since the entropy of the noise is constant, our objective is to maximize the mutual information between $\mathbf{X}$ and the component choice, $\mathbf{A}$. We frame this in terms of trying to infer the true component choice, $\mathbf{A}$, from the observed sample, $\mathbf{X}$. To this end, we consider the prior distribution of the vertices' distances to the sample, $D = \|\mathbf{X} - \mathbf{B}\|_2^2$, where $\mathbf{B}$ represents a candidate component choice and is distributed identically to $\mathbf{A}$. Noting that, $D = \sum_i D_i$ where $D_i = (A_i + \mathcal{E}_i - B_i)^2$, we have that

$$P(\bar{D} = d) \approx \exp\left(-NI(d)\right)$$

and the likelihood for $\bar{D}$ is

$$P(\mathbf{X}|\bar{D} = d) = \exp\left(-\frac{1}{2\sigma^2}Nd\right)$$

Thus the posterior mode of $\bar{D}$ is attained where $N(I(d) + \frac{d}{2\sigma^2})$ is minimized, which occurs when $I'(d) = -\frac{1}{2\sigma^2}$. At this point, the prior probability of the posterior mode will be $\exp(-NI(d))$, meaning that the observation delivers $NI(d)$ bits of information about the component choice, or $I(d)$ bits per sample.

$I(d)$ had a closed form expression, although its derivation is a somewhat formidable piece of algebra. We begin by defining the random variable $C_i = A_i - B_i$, which is also a discrete random variable taking on the values of $d_{ij} = \mu_i - \mu_j$ with probabilities $\rho_{ij} = \pi_i \pi_j$.

Solving for I in terms of its derivative, and setting it to $-\frac{1}{2\sigma^2}$, we have that

$$\mathbf{I}\left(\theta = -\frac{\mathbf{1}}{\mathbf{2\sigma^2}}\right) = \frac{\mathbf{1}}{\mathbf{2}}\ln(\mathbf{2}) - \frac{\mathbf{1}}{\mathbf{4}} - \ln(\mathbf{Z}) - \frac{\mathbf{1}}{\mathbf{8\sigma^2}}\frac{\mathbf{Y}}{\mathbf{Z}}$$

where

$$\mathbf{Z} = \sum_{\mathbf{i,j}} \rho_{\mathbf{ij}} \exp\left(-\frac{\mathbf{d_{ij}^2}}{\mathbf{4\sigma^2}}\right)$$

and

$$\mathbf{Y} = \sum_{\mathbf{i,j}} \rho_{\mathbf{ij}} \mathbf{d_{ij}^2} \exp\left(-\frac{\mathbf{d_{ij}^2}}{\mathbf{4\sigma^2}}\right)$$

.

The final term may be recognized as the expected squared distance of C_i under a Boltzmann distribution.

6.3 User Parameters

6.3.1 The Likelihood Model

We model our observations of the user's signal as being subject to two sources of noise: an outlier probability, ρ, that the signal was unintended, and additive noise on intended signals. We model the unintended signals as uniformly distributed, and the noise as a Gaussian. (We do not require that the noise have zero mean, since the user may be consistently late or early, relative to the intended time.) Hence our likelihood model for the received signal is a uniform-gaussian mixture model, characterized by the outlier probability, ρ, and the parameters of the noise, μ_ϵ, σ_ϵ.

6.3.2 Directional Statistics

Chapter 7

Render Stimulus

7.1 Circular Timers

In this step we represent the nodes' phases to the user. Dotter does this by drawing a small circular "timer" for each node.

We justify this choice with reference to the concept of "local attention".

One visually simpler choice would be to let a "selection" bar descend the screen from top to bottom, and to adjust nodes' vertical positions to reflect their phases. This approach resembles that row and column selection method of many communication boards.

Although this is less cluttered, it requires the use of global attention. By letting each node carry its own timer, the user does not need to be aware of any part of the screen outside of their foveal vision.

Chapter 8

Detect Response

Dotter requires the precise timing of the user's signal. Because the user's signal's variance is often as little a 20 milliseconds, and signal detection frames might be spaced as far apart as 30 milliseconds, the noise introduced by the signal detection process can be on the order of the noise introduced by the user.

8.1 Blink Detection

This frame frequency problem arises when the user's signal is a blink. A typical laptop webcam might capture 40 frames per second. If we use a simple threshold to detect a blink, this will introduce on the order of 20 milliseconds of noise. To resolve this problem we should use interpolation. The blink signal can be linearly interpolated between the frame before and the frame after the threshold was crossed to estimate the precise time of the blink. In post-hoc analysis making this change reduced about 3 milliseconds of noise when my blink standard deviation was about 20 milliseconds.

Further noise reductions should be made through a simple linear regression of features of the blink signal against the correct blink times. This can remove another millisecond of noise.

8.2 Keypress Detection

For an in-browser implementation, the `keydown` even should be preferred at the time should be read directly from the `event.timeStamp` field which provides the precise time that the keydown even was received by the browser.

Chapter 9

Determine Most Likely Break Nodes

One challenge of Dotter is that likelihoods are received with reference to a character-based trie, but priors are received with reference to a token-based trie. A considerable portion of the implementation is devoted to closing that gap.

In the chapter 4 we saw how to identify the character sequence prefixes with the highest posterior probabilities so that we could display them to the user. Here our task is to identify the token sequence prefixes with the highest posterior probabilities, so that we can query the language model for predictions. Just as we solved that problem with a character-based trie, so here we will solve it with a token-based trie. A token-based trie has a very high radix, equal to the model's vocabulary size, e.g., 50,257 for GPT-2.

9.1 An Upper Bound for the Likelihood of a Token Sequence

In the character-based trie (see chapter 2), calculating the likelihoods was trivial, but owing to the token-based nature of the language model, calculating the priors of a character sequence was somewhat involved. Here the situation is reversed. Calculating the prior of a token sequence is trivial, since that is directly what the language model returns, while calculating the likelihood of a token sequence prefix is difficult.

For any given token prefix A, and any token prefix sequence paritition set T (e.g. the set of all tokens, i.e., all token sequences of length 1), the likelihood of A is given by,

$$P(D|A) = \sum_{t \in T} P(D|A+t)P(A+t|A)$$

However, this poses a problem since we assume that $P(A+t|A)$ is not known. We are seeking to evaluate the likelihood of A in order to decide the priority of querying the language model for its next token, but in order to calculate this likelihood, we must already know the prior probabilities of the next token conditioned on A! Thus we are in a catch-22 and must resort to using a heuristic for the likelihood.

Since the likelihood of A is a weighted sum of its descendants' likelihoods, it must be bounded by the minimum and maximum of its descendants' likelihoods.

$$\min_{t \in T} P(D|A+t) \leq P(D|A) \leq \max_{t \in T} P(D|A+t)$$

Since we don't want to rely upon our current model of the prior, we require choose T so that all the descendants $A+t$ have a valid likelihood, i.e., so that they have never been a parent of a visible node.

We will prioritize token prefixes by their upper bound on the likelihood. Thus it is necessary to know, for a given node, the maximum likelihood of any descendant which has this node as a token ancestor. This is referred to as the `max_token_descendant_likelihood` of the node.

Maintaining this value during likelihood updates is discussed in chapter **??**.

9.2 Descending the Token Trie

Our choice of upper bound has the property that it strictly decreases with depth. This is because if a given node, x, is a token descendant of B, and B has A as a token ancestor, then x must be a token descendant of A as well and thus `A.max_token_descendant_likelihood` $\geq$ `B.max_token_descendant_likelihood`.

We make a priority-first search of the token trie, visiting the node with the highest upper bound on its posterior probability first. When we visit a node about which we have not yet queried the language model, we query the language model (chapter 10) about this node. The next token probabilities are referred to as the "LLM Result" which can then be consumed to update the character-based trie with a prior update (chapter **??**). Our descent of the token trie is not interrupted by the prior update, but we will need to restart our descent from the root after any likelihood update.

When we visit a node we must add its children to the priority queue. However, the radix is very high, equal to the vocabulary size (e.g., $V = 50,257$ for GPT-2). Thus we only add the children with the large posterior upper bounds to the queue. In implementation we use $K = 100$. To identify these one hundred children with the highest posterior upper bounds, we must calculate the posterior upper bound for all children. The language model itself gives us an array of length V for the prior probabilities of the children. Our task is to construct the array of likelihood upper bounds. To do so, we descend the character-based trie from the node in the character-based trie which corresponds to our current node in the token-based trie.

We descend the character-based trie until one of two conditions are met:

1. The descendant has a valid likelihood, i.e., it has never been a parent of a visible node. In this case, we know that this likelihood is the likelihood of all possible child token sequences. We set the relevant alphabetical range of the likelihood array to this likelihood. (E.g. if our node is 'the', and this descendant with a valid likelihood is 'the ho', we must set the likelihood array to this descendant's likelihood for every token in the range ' ho' to ' hp' such as the tokens ' horse', ' home', ' hound', etc.)
2. It is impossible for deeper descendants to be direct token children of our root node. I.e., the descendant does not extend our node by a strictly partial token prefix. In this case, we are at a direction token sequence child, 'B', of our root node, 'A'. For the specific element of the likelihood array corresponding to the last token of B, we set the likelihood to the maximum descendant likelihood of B which could have B as a token ancestor, i.e., `B.max_token_descendant_likelihood`.

Chapter 10

LLM Query

This step is straightforward. We call a model to find the distribution over the next token for a given token sequence. The only challenge is to correctly maintain the model's Key-Value Cache.

10.1 Maintaining a Key-Value Cache Trie

Traditionally, when we call a language model to generate a sequence of tokens, our Key-Value Cache is a simple list that grows as the sequence of tokens is extended. In our case, we must use a trie to store the Key-Value Cache.

Chapter 11

Mastering Dotter

11.1 The Cost of Incorrect Signals

Expertise in Dotter is about accuracy in three ways: (1) The user must synchronize their signals with the target timers. (2) The user must accurately select the correct prefix. (3) The user must not make involuntary signals (e.g. involuntary blinks).

From the program's perspective, there is virtually no difference between the user selecting the wrong prefix and making an involuntary signal. Both are considered as incorrect signals and both are assumed to have random phase. If the user intended the correct timer and was merely imprecise in synchronizing their signal, we do not consider this to be an "incorrect" signal. [1].

Here we will quantify the cost of incorrect signals.

UNFINISHED

The user's speed can be calculated from their performance parameters. Here we consider the effect of ρ, the probability of unintended signals. Given an observation, D, the gain in logits for the true hypothesis h is $\ln P(D|h) - \ln P(D|\neg h)$. When the posterior probability of h is small, this gain is equal to the gain in the log posterior probability of h. Let r denote the hypothesis that the signal was unintended, so that $P(r) = \rho$.

We state the following facts:

$$P(D|\neg h, r) = P(D|\neg h, \neg r) = P(D|\neg h)$$

$$P(D|\neg h) \approx P(D), \quad \text{when } P(h) \approx 0$$

$$P(D|h, r) = P(D), \quad \text{assuming that } D,\ h \text{ are independent conditioned on } r$$

The likelihood under the true hypothesis is

$$P(D|h) = \rho P(D|h, r) + (1 - \rho)P(D|h, \neg r)$$

We consider the case where $P >> \sigma$ so that the components of the mixture are well-separated. This means that the contribution of the false component is negligible and that we can approximate the likelihood $P(D|h)$ piecewise as follows:

$$P(D|h) \approx \begin{cases} P(D|h, r)P(r) & \text{if } D \sim P(D|h, r) \\ P(D|h, \neg r)P(\neg r) & \text{if } D \sim P(D|h, \neg r). \end{cases}$$

That is, when D is likely drawn from the unintended distribution, $P(D|h) \approx P(D|h, r)P(r)$. When D is likely drawn from the intended distribution, $P(D|h) \approx P(D|h, \neg r)P(\neg r)$.

[1] We may safely assume that an involuntary signal has random phase. As for an incorrectly selected prefix, if we assume (perhaps unreasonably) that the incorrect prefix was randomly selected from all prefixes, and we further assume (legitimately) that we have uniformly distributed the prefixes' timers, then such a signal has random phase as well.

$$E_{D|r,h}\left[\ln P(D|h)\right] \approx E_{D|r,h}\left[\ln\Big(P(D|h,r)P(r)\Big)\right]$$
$$= \ln\rho - h(D|h,r)$$

and similarly

$$E_{D|\neg r,h}\left[\ln P(D|h)\right] \approx \ln(1-\rho) - h(D|h,\neg r)$$

The expected log likelihood of the data under the true hypothesis is

$$\begin{aligned}
E_D\left[\ln P(D|h)\right] &= \rho E_{D|r,h}\left[\ln P(D|h)\right] + (1-\rho)E_{D|\neg r,h}\left[\ln P(D|h)\right] \\
&\approx \rho(\ln\rho - h(D|h,r)) + (1-\rho)(\ln(1-\rho) - h(D|h,\neg r)) \\
&= -H_2(\rho) - \rho h(D|h,r) - (1-\rho)h(D|h,\neg r) \\
&= -H_2(\rho) - \rho h(D) - (1-\rho)h(D|h,\neg r), \quad \text{assuming } D|r,\ h|r, \text{ independent}
\end{aligned}$$

Where H_2 is the binary entropy function.

The likelihood of the alternative hypothesis is

$$E_{D|h}\left[\ln P(D|\neg h)\right] \approx E_{D|h}\left[\ln P(D)\right]$$

To simplify this term we use its expectation over h. Recalling the fact that,

$$E_a\left[E_{b|a}[f(b)]\right] = E_b[f(b)]$$

, we find that this term's expectation over h is $-h(D)$.

So the total expected gain in the logits of our hypothesis is

$$E_{D|h}[\ln P(D|h) - P(D|\neg h)] \approx -H_2(\rho) + (1-\rho)h(D) - (1-\rho)h(D|h,\neg r)$$

In our particular model, D is uniformly distributed over an interval of length P and the entropy of D conditional on h is just the entropy of the gaussian noise. Denoting the latter term by H_σ, we have,

$$E_{D|h}[\ln P(D|h) - P(D|\neg h)] \approx -H_2(\rho) + (1-\rho)(\ln P - H_\sigma)$$

This has a nice interpretation: we gain $\ln P - H_\sigma$ bits of information per intended signal, but the presence of unintended signals makes us pay a penalty in two ways: only $(1-\rho)$ of the time is our signal intended, and additionally we must expend $H_2(\rho)$ bits to learn whether the signal was intended.

Unfortunately the singularity of the binary entropy function at $\rho = 0$ becomes a practical problem for fast text entry. For example, a seemingly low outlier rate of $\rho = 0.03$ imposes the relatively high cost of $H_2(0.03) \approx 0.19$ bits in addition to the 3% loss that would be naively expected. Under reasonable parameters for an intermediate user (e.g. $\sigma = 25ms$, $P = 700ms$), these combined terms amount to a loss of more than 10% of the channel's capacity. Thus to achieve expert entry speeds, it is necessary to make unintended signals very rarely.

Bibliography

[1] Artemy Kolchinsky and Brendan Tracey. Estimating mixture entropy with pairwise distances. *Entropy*, 19(7):361, July 2017.

[2] David J. Ward, Alan F. Blackwell, and David J. C. MacKay. Dasher—a data entry interface using continuous gestures and language models. In *Proceedings of the 13th Annual ACM Symposium on User Interface Software and Technology*, UIST '00, pages 129–137, New York, NY, USA, 2000. ACM.